POLITECNICO DI BARI

Dipartimento di Ingegneria Elettrica e dell'Informazione
Corso di Laurea triennale in Ingegneria Informatica e dell'Automazione

---

Tesi di Laurea in
RETI DI TELECOMUNICAZIONI

PROGETTAZIONE E ANALISI DELLE PRESTAZIONI DI UN SOFTWARE DEFINED FIREWALL PER COMPETIZIONI ATTACCO/DIFESA

Relatore
Chiar.mo Prof. Gennaro Boggia

Correlatore
Dott. Ing. Daniele Pugliese

Laureando
Domingo Dirutigliano

Anno Accademico 2024 - 2025

Abstract

L'aumento esponenziale di attacchi cyber, fenomeni di data breach e violazioni della privacy ha elevato la cybersecurity a priorità strategica a livello globale, determinando una crescente domanda di professionisti specializzati, capaci di fronteggiare scenari dinamici attraverso competenze tecniche multidisciplinari e approcci innovativi. Tra le metodologie di addestramento delle nuove generazioni di esperti spicca la gamification applicata alle competizioni Capture The Flag (CTF). Queste sfide simulano scenari reali, fornendo un laboratorio pratico per acquisire competenze tecniche in contesti operativi, sviluppare capacità di problem solving under pressure e comprendere le dinamiche degli attacchi moderni. Inoltre, esistono competizioni organizzate in tutto il mondo che riuniscono e si rivelano sfidanti per la comunità di hacker e professionisti della sicurezza informatica, spesso promosse direttamente da grandi aziende. In questo quadro si inserisce il presente lavoro, finalizzato alla progettazione e sviluppo di un modulo di un Software Defined Firewall (SDFW) specializzato per CTF di tipologia Attack/Defense (A/D). La soluzione proposta integra funzionalità avanzate di analisi del traffico in *real-time* e meccanismi adattivi di filtraggio, permettendo di scrivere filtri direttamente in linguaggio Python. Il modulo si dimostra altamente configurabile e versatile, caratteristiche essenziali per questo tipo di competizioni. L'architettura è stata progettata al fine di garantire un elevato throughput nonostante l'uso di un linguaggio interpretato, grazie all'uso di tecniche di parallelizzazione e all'implementazione eseguita in linguaggio C++.

Indice

Introduzione 1

1 Competizioni CTF Attack/Defense 5
1.1 Descrizione dell'infrastruttura 5
1.1.1 Il Gameserver e i Checker 6
1.1.2 I servizi, le flag e i flag store 6
1.1.3 I Flag ID 7
1.1.4 Calcolo del punteggio 7
1.1.5 Rete di gara 9
1.2 Svolgimento della competizione 11
1.3 Strumenti utili durante la competizione 11
1.3.1 Traffic Analyzer 12
1.3.2 Attacker & Submitter 13
1.3.3 Proxy e Firewall 14
1.4 Esempio di un servizio vulnerabile: PCSS 15

2 Firegex 19
2.1 Perché nasce Firegex? 20
2.2 Confronto con altre soluzioni 21
2.3 Integrazione con il networking su Linux 22
2.3.1 Netfilter Tables 23
2.3.2 Netfilter Queue Module 24
2.4 Architettura 25
2.4.1 Frontend e Backend 26
2.4.2 Moduli C++ 26
2.5 Funzionalità 27
2.5.1 Modulo nfregex 27
2.5.2 Modulo firewall 28
2.5.3 Modulo porthijack 28
2.5.4 Modulo nfproxy 28

3 Progettazione e sviluppo del modulo nfproxy 31
3.1 Scelte progettuali 32
3.2 Architettura 34
3.3 Elaborazione parallelizzata dei pacchetti 35
3.3.1 A Per-Interpreter Global Interpreter Lock 35
3.3.2 Bilanciamento del carico 38
3.4 Parsing dei pacchetti L3 39
3.5 Gestione dei pacchetti TCP 41
3.6 Modifica dei pacchetti 42
3.6.1 Cambio di dimensione del payload 43
3.6.2 Modifica di segmenti non ordinati 45
3.7 Elaborazione dei pacchetti HTTP 46
3.7.1 Supporto agli algoritmi di compressione in HTTP 47
3.7.2 Supporto alle websocket 47
3.8 Sintassi e gestione dei filtri python 48
3.8.1 Gestione dei buffer di memoria 50
3.9 Proxy di simulazione 50

4 Testing e Benchmarking 53
4.1 Integrazione dei test 53
4.2 Benchmark nfproxy 54
4.3 Benchmark sul sistema di balancing 56

Conclusioni 59

Note 61

Ringraziamenti 63

Bibliografia 65

Elenco delle figure

1.1 Scoreboard di una simulazione realizzata con CTFBox[10]. 8
1.2 Modello della rete di gara di Cyberchallenge[9]. 10
1.3 Logo di Caronte[14]. 12
1.4 Interfaccia di Caronte[14]. 13
1.5 Logo di ExploitFarm. 14
1.6 Interfaccia di ExploitFarm. 14
1.7 Logo di Firegex. 15

2.1 Interfaccia grafica di Firegex. 19
2.2 Grafico degli hooks di Netfilter Tables. 23
2.3 Schema generale dell'architettura di Firegex. 25
2.4 Interfaccia di dettaglio del servizio nel modulo nfproxy. 29

3.1 Architettura del modulo nfqueue. 34
3.2 Python Global Interpreter Lock[50]. 36
3.3 Esempio di una possibile problematica nella traduzione di ACK e SEQ implementata. 45
3.4 Simulazione di nfproxy tramite il comando fgex. 51

4.1 Grafico Whisker sulle misure di throughput di nfproxy. 55
4.2 Grafico Whisker sulle misure di throughput di nfproxy e nfregex. . . . 56
4.3 Benchmark di nfproxy con load balancing personalizzato (1 vs 8 thread). 57

Elenco dei codici

1.1 Algoritmo di calcolo del valore della flag in CyberChallenge[9]. 6
1.2 Algoritmo di calcolo del punteggio in CyberChallenge[9]. 9
1.3 Interfaccia CLI di PCSS (da CTFBox[10]). 16
1.4 Funzione vulnerabile a Local File Inclusion nel servizio PCSS (da CTFBox[10]). 16
1.5 Funzione per eseguire un attacco tramite un token JWT malevolo su PCSS (da CTFBox[10]). 17

2.1 One-Command per l'esecuzione di Firegex. 27

3.1 Inizializzazione di un processo consumatore con GIL indipendente in nfproxy. 37
3.2 Funzione di hashing per stream_id (valida per IPv4/IPv6) in nfproxy. 38
3.3 funzione nfqueue_hash dal kernel Linux (semplificata) (versione 6.14-rc7). 39
3.4 Costruttore di PktRequest ed elaborazione del pacchetto di rete in firegex. 40
3.5 Calcolo dell'offset cumulativo nello stream TCP da correggere in modifica. 43
3.6 Algoritmo di traduzione di ACK e SEQ numbers a seguito di mangle dei pacchetti. 44
3.7 Esempio di un filtro python per il modulo nfproxy. 48
3.8 Esempio di configurazione di un filtro nfproxy per l'ottimizzazione del consumo di memoria. 50
3.9 Comando per avviare il simulazione di nfproxy integrato nella CLI di firegex. 50

4.1 Comandi per l'esecuzione dei benchmark su nfproxy e nfregex 55
4.2 Algoritmo di simulazione del carico usato nei benchmark multi-thread di nfregex. 57
4.3 One-Command tramite cui eseguire la POC della CVE-2022-36946. . 61

Acronimi

A/D Attack/Defense

ACK Acknowledgment Number

API Application Programming Interface

CLI Command Line Interface

CPU Central Processing Unit

CTF Capture The Flag

DoS Denial of Service

FIFO First In First Out

GHCR GitHub Container Registry

GIL Global Interpreter Lock

HTTP Hypertext Transfer Protocol

ICMP Internet Control Message Protocol

IP Internet Protocol

IPv4 Internet Protocol version 4

IPv6 Internet Protocol version 6

JSON JavaScript Object Notation

JWT JSON Web Token

LFI Local File Inclusion

MITM Man In The Middle

NAT Network Address Translation

nfproxy Netfilter Simulated Proxy

nfqueue Netfilter Queue

nfregex Netfilter Regex Filter

nftables Netfilter Tables

NGFW Next Generation Firewall

NOP No Operation

PCRE Perl Compatible Regular Expressions

PDU Protocol Data Unit

PEP Python Enhancement Proposal

POC Proof Of Concept

porthijack Netfilter Port Hijack

PyPI Python Package Index

regex Regular Expression

SDFW Software Defined Firewall

SEQ Sequence Number

SLA Service Level Agreement

TCP Transmission Control Protocol

UDP User Datagram Protocol

UFW Uncomplicated Firewall

VPN Virtual Private Network

Introduzione

L'era digitale, caratterizzata da una connettività globale senza precedenti, ha portato con sé una crescente esposizione a minacce informatiche sofisticate. Attacchi ransomware, violazioni di dati sensibili, phishing avanzato e exploit zero-day rappresentano oggi una realtà quotidiana, con impatti devastanti su infrastrutture critiche, istituzioni finanziarie e diritti individuali. Secondo il Global Risk Report 2025 del World Economic Forum[1], la cybersecurity si colloca tra i primi cinque rischi globali in termini di probabilità e gravità, sottolineando l'urgente necessità di rafforzare la resilienza dei sistemi e formare professionisti capaci di anticipare, contenere e neutralizzare queste minacce.

In questo contesto, la domanda di esperti con competenze tecniche multidisciplinari: dalla crittografia all'analisi forense, dalla sicurezza di rete alla psicologia degli attaccanti, ha superato l'offerta creando un divario che richiede approcci innovativi alla formazione. Le tradizionali metodologie didattiche, spesso teoriche e frammentate, si rivelano insufficienti per preparare figure capaci di operare in scenari dinamici, dove la rapidità decisionale e la creatività strategica sono determinanti.

Un ruolo chiave in questa transizione formativa è svolto dalle competizioni CTF, che combinano gamification e simulazione di scenari reali per offrire un addestramento immersivo. Queste sfide, strutturate in categorie come jeopardy, A/D o Boot-to-Root, ricreano ambienti controllati in cui i partecipanti devono identificare vulnerabilità, sfruttare exploit, difendere servizi e analizzare tracce malevole. Non solo favoriscono l'acquisizione di competenze tecniche, ma allenano il *problem solving under pressure*, la collaborazione in team e la capacità di adattamento a tattiche in continua evoluzione. Eventi internazionali come il Google CTF[2], patrocinato da aziende leader del settore, attirano migliaia di partecipanti, trasformandosi in vetrine per il reclutamento di talenti e laboratori per testare strumenti all'avanguardia.

L'obiettivo in queste competizioni è rubare la flag: una stringa di testo segreta, con un formato ben riconoscibile, ottenibile sfruttando vulnerabilità volutamente esposte nei sistemi. La flag rappresenta pertanto la prova univoca che attesta la

[1] Global Risks Report 2025: `https://www.weforum.org/publications/global-risks-report-2025/`

[2] `https://capturetheflag.withgoogle.com/`

risoluzione della sfida da parte dei partecipanti. Queste competizioni costituiscono un ottimo strumento per approfondire le proprie competenze nel settore anche per i più esperti, che spesso ricoprono sia il ruolo di giocatori e talvolta di organizzatori, alimentando un ciclo virtuoso di apprendimento, innovazione e crescita collettiva all'interno dell'ecosistema cybersecurity[3].

Particolare rilevanza assumono le competizioni di tipo A/D, in cui i team devono simultaneamente proteggere i propri sistemi dagli avversari e compromettere quelli altrui. Questa modalità richiede una comprensione profonda delle dinamiche offensive e difensive, oltre alla capacità di implementare contromisure in tempo reale.

Questa tesi tratta la progettazione e lo sviluppo di un SDFW orientato alle competizioni CTF di tipologia A/D, le quali richiedono tool flessibili in grado di adattarsi dinamicamente alle varie fasi della competizione. Verranno analizzate le funzionalità attualmente presenti nel progetto, i punti critici e le possibili migliorie per aumentarne l'efficienza e le prestazioni. In particolare, l'attenzione si concentrerà su una nuova funzionalità sviluppata che consente l'applicazione di filtri in modo totalmente trasparente dal punto di vista dell'applicativo, che mira a minimizzare l'impatto sulla rete, e permette di definire le regole di filtraggio utilizzando istruzioni in linguaggio Python. Il firewall oggetto di questa tesi è Firegex[4], sviluppato per il team del Politecnico di Bari[5] in occasione della competizione nazionale di Cyberchallenge[6] del 2022. In tale contesto è nato lo stesso team CTF che ne ha supportato lo sviluppo, Pwnzer0tt1[7]. La nuova funzionalità implementata, denominata `Netfilter Simulated Proxy (nfproxy)`, simula il funzionamento di un proxy tradizionale per il filtraggio del traffico. Il suo funzionamento e caratteristiche tecniche saranno oggetto del Capitolo 3.
La struttura del documento è organizzata come segue:

- Nel Capitolo 1 verranno forniti i dettagli relativi alle competizioni CTF A/D, descrivendo l'infrastruttura tipica di gara, il funzionamento dei componenti principali e gli strumenti comunemente utilizzati dai partecipanti;
- Il Capitolo 2 presenterà Firegex analizzandone le motivazioni alla base dello sviluppo, il confronto con soluzioni esistenti, l'architettura interna e le principali funzionalità offerte;

[3] `https://www.hackthebox.com/blog/what-is-ctf/`

[4] `https://github.com/Pwnzer0tt1/firegex`

[5] `https://www.poliba.it`

[6] `https://cyberchallenge.it/`

[7] `https://pwnzer0tt1.it/`

- Il Capitolo 3 si focalizzerà sulla progettazione e sviluppo del modulo `nfproxy`, approfondendo le scelte architetturali, le problematiche nell'elaborazione parallela dei pacchetti e le soluzioni implementate per la gestione del traffico Transmission Control Protocol (TCP) e Hypertext Transfer Protocol (HTTP);
- Il Capitolo 4 descriverà la metodologia adottata per i test di funzionalità e performance, inclusi i benchmark comparativi e i risultati ottenuti;
- Infine, nelle Conclusioni ci saranno considerazioni finali su possibili sviluppi futuri ed in particolare verrà citata una vulnerabilità nel kernel Linux emersa durante lo sviluppo del progetto.

Capitolo 1

Competizioni CTF Attack/Defense

Le competizioni CTF di tipo A/D sono una tipologia di competizioni in cui i partecipanti devono difendere i propri servizi e attaccare i servizi degli avversari, in un contesto di gara estremamente dinamico e competitivo. In queste competizioni partecipano diversi team, ognuno dei quali è associato a una macchina, solitamente Linux, con in esecuzione un insieme di servizi, ciascuno dei quali presenta vulnerabilità[8]. I servizi sono accessibili tramite la rete di gara e sono replicati su tutte le macchine.

1.1 Descrizione dell'infrastruttura

L'infrastruttura di una competizione CTF Attack/Defense è composta da diversi componenti chiave che interagiscono in modo sinergico. Possiamo individuare tre elementi principali che costituiscono l'infrastruttura di gara:

- Il **Gameserver**, cuore centrale del sistema che gestisce la competizione;
- I **Checker**, responsabili del monitoraggio continuo della rete e dei servizi;
- Le **VM dei team**, che ospitano i servizi vulnerabili.

Il funzionamento di ciascuno di questi componenti è approfondito nei paragrafi successivi, evidenziando inoltre tutte le peculiarità e le funzionalità utili allo svolgimento della competizione.

[8] `https://faustctf.net/information/attackdefense-for-beginners/`

1.1.1 Il Gameserver e i Checker

Al centro di tutta la competizione c'è il gameserver, entità (che può essere composta da uno o più host) che si occupa della gestione della competizione, eseguendo una serie di azioni e fornendo diversi servizi descritti in questo capitolo, essenziali per lo svolgimento della gara stessa.

La competizione è strutturata in tick (o round), ovvero intervalli di tempo in cui ciclicamente il gameserver assegna punti ai team in base alle azioni compiute dagli stessi, e monitora i servizi tramite i checker. Questo tempo può variare da competizione a competizione. I checker sono dei bot che si occupano di supervisionare lo stato della gara eseguendo diverse azioni come:

- Verificare se il servizio è funzionante e raggiungibile, andando a connettersi ed eseguendo azioni di normale utilizzo, assicurandone anche l'integrità;
- Utilizzare il servizio per memorizzare al suo interno una flag, inserita come informazione sensibile normalmente non accessibile seguendo i normali workflow del servizio;
- Verificare che le flag inserite nei tick precedenti siano ancora raggiungibili tramite un normale accesso autorizzato, e che pertanto non siano state modificate o rimosse dal team possessore di quella macchina.

In generale, rimuovere vecchie flag, avere il servizio non raggiungibile e non avere la possibilità di inserirne di nuove, sono azioni che vanno ad invalidare la normale attività del servizio: il gameserver, rilevate queste problematiche, lo andrà a considerare come totalmente o parzialmente inaccessibile o manomesso, in base alle scelte degli organizzatori. Si evidenzia come i checker non sfruttino mai le vulnerabilità dei servizi, ma si limitino al loro corretto utilizzo.

1.1.2 I servizi, le flag e i flag store

Ogni flag viene generata e inserita dai checker nel servizio dei vari team a ogni tick, con una flag sempre differente dalle altre e con una durata massima di validità (e.g., 5 tick): in questo intervallo di tempo è ancora possibile rubare e consegnare la flag al gameserver ottenendo punti. Le flag infatti una volta ottenute vanno consegnate al gameserver, che tramite un calcolo che può variare in diverse competizioni, assegna punti al team che l'ha consegnata come mostrato nel Codice 1.1.

```
scale = 15 * sqrt(5)
norm = ln(ln(5)) / 12
offense_points[flag] = scale / (1+exp((sqrt(score[attacker][service]) -
↪  sqrt(score[victim][service]))*norm))
defense_points[flag] = min(victim_score, offense_points)
```

Codice 1.1: Algoritmo di calcolo del valore della flag in CyberChallenge[9].

Il defense point è il punteggio sottratto al team da cui è stata rubata la flag come parte del punteggio di difesa descritto successivamente.

All'interno della competizione spesso possono esserci dei team di 'prova' chiamati No Operation (NOP) team: questi sono semplicemente macchine non associate a nessun gruppo reale di persone, che offrono i servizi della gara, al solo scopo di renderli usufruibili per eseguire penetration testing. Il loro utilizzo è particolarmente utile ai partecipanti per provare gli attacchi in maniera quanto più fedele e realistica possibile, eseguendo l'attacco su una replica originale del servizio, e non su una potenzialmente soggetta alle modifiche e alle *patch* del team avversario. Non tutte le competizioni prevedono la presenza dei NOP team. Si noti che consegnare le flag del proprio team o quelle appartenenti ai NOP team non permette di guadagnare alcun punteggio, e ciò è possibile poiché il gameserver ha traccia di tutte le flag, di chi le consegna e delle macchine in cui sono state originariamente inserite.

Una caratteristica importante da sottolineare riguardo ai servizi di cui si è parlato finora è che non è sempre vero che per ognuno di essi ci sia una sola flag per tick e una sola vulnerabilità per singolo tick. Infatti, ogni servizio ha usualmente multiple vulnerabilità e flag per tick al suo interno: i checker che si occupano di inserire le flag potrebbero inserirne diverse, usando diverse parti o impostazioni del servizio. In caso di servizi con più flag, diremo che questi conterranno diversi flag store. Ciò detto fino ad ora è pertanto valido per ogni flag store, e non per il servizio in generale. Si evidenza l'esistenza un set di checker differente per ogni flag store.

1.1.3 I Flag ID

Il gameserver potrebbe rilasciare delle informazioni aggiuntive per ogni flag store, come ad esempio un username, che di per se non va direttamente a permettere l'accesso protetto alla flag, ma da un'informazione utile agli attaccanti al fine di poterla individuare più facilmente all'interno del servizio stesso. Questa informazione aggiuntiva è chiamata Flag ID e viene rilasciata per ogni team, per ogni flag store, e per ogni flag ancora valida nel momento in cui si visita l'Application Programming Interface (API) esposta dal gameserver, che le espone pubblicamente per tutti i partecipanti. Non è possibile manomettere i Flag ID, in quanto sono informazioni direttamente rilasciate dagli organizzatori, che comunque in assenza di vulnerabilità nei servizi, risultano insufficienti per ottenere flag.

1.1.4 Calcolo del punteggio

A ciascun flag store in gara viene associato un punteggio, che riflette la capacità del team di sfruttare e proteggere le relative vulnerabilità. Il calcolo del punteggio totale si ottiene tradizionalmente mediante una semplice sommatoria dei punteggi parziali. Il punteggio attribuito a un singolo store è inizialmente determinato da un valore base, soggetto a variazioni durante la competizione in relazione alle prestazioni

del team. Tale valore può essere incrementato o ridotto in base a tre componenti aggiuntive:

- **Punteggio di Service Level Agreement (SLA)**: misura la disponibilità del servizio, calcolata in proporzione al tempo di effettiva operatività. Questo indicatore, generalmente in percentuale, viene applicato al punteggio complessivo dello store. La scelta di attribuirgli un ruolo predominante mira a scoraggiare strategicamente la disattivazione volontaria dei servizi per evitare attacchi, penalizzando severamente tale approccio. Di conseguenza, il mantenimento dell'operatività del servizio (pur nella sua condizione vulnerabile) assume un'importanza critica nella competizione;
- **Punteggio di attacco**: determinato dal numero di invii corretti di flag effettuati dal team. Il valore attribuito a ciascuna flag può variare in funzione della complessità dell'attacco e della tempestività con cui il servizio è stato compromesso rispetto agli altri team;
- **Punteggio di difesa**: tipicamente negativo, poiché quantifica l'inefficacia delle contromisure, è calcolato in base al numero di flag sottratte al team da parte degli avversari. Tale componente tende a controbilanciare, o persino ridurre, il punteggio di attacco relativo allo stesso flag store, potendolo portare anche al di sotto del valore base iniziale. La sua entità è modulata da criteri analoghi a quelli usati per il punteggio di attacco. Per mitigarne gli effetti, risulta essenziale applicare interventi correttivi (*patch*) che impediscano nuovi attacchi, garantendo al contempo la piena operatività del servizio e l'integrità dei dati archiviati (comprese le flag precedenti, la cui perdita comporterebbe un deterioramento del punteggio di SLA).

Figura 1.1: Scoreboard di una simulazione realizzata con CTFBox[10].

È possibile analizzare come è effettuato il calcolo dal Codice 1.2 e come è visualizzato ai giocatori nella scoreboard in Figura 1.1 nel simulatore A/D CTFBox[10].

```
# --- Score totale per ogni flag store ---
# Punteggio base per ogni flag store
score[team][service] = 5000

# Somma dei punteggi di attacco
for flag in stolen_flags[team][service]:
    score[team][service] += offense_points[flag]

# Sottrazione dei punteggi di difesa
for flag in lost_flags[team][service]:
    score[team][service] -= defense_points[flag]

# Calcolo dello score finale del team
total_score[team] = 0
for service in services:
    # Calcolo dello SLA per il servizio
    sla[team][service] = ticks_up[team][service] / ticks[team][service]
    # Limitazione del punteggio minimo dello score a 0
    score[team][service] = max(0, score[team][service])
    # Somma del punteggio del servizio con SLA applicata
    total_score[team] += score[team][service] * sla[team][service]
```

Codice 1.2: Algoritmo di calcolo del punteggio in CyberChallenge[9].

1.1.5 Rete di gara

Per questa analisi si prende in considerazione la rete di gara in utilizzo per le competizioni nazionali di Cyberchallenge[9], sottolineando come a grandi linee la configurazione di rete è spesso simile, ma potrebbe variare in base alle competizioni e agli organizzatori.

Come si può vedere nella Figura 1.2, la rete di gara è composta da diversi elementi:

- La rete dedicata agli organizzatori, dove abbiamo il gameserver (potenzialmente formato da un insieme di host) ed eventuali ulteriori host al servizio degli organizzatori;
- La sottorete delle macchine dei team, dove si ha accesso ssh alla propria macchina con i propri servizi vulnerabili;

[9] https://ad.cyberchallenge.it/rules/

- Una sottorete (spesso realizzata tramite Virtual Private Network (VPN)) per ogni team, dove sono connessi gli host dei partecipanti con i quali è possibile interagire direttamente con la rete di gara.

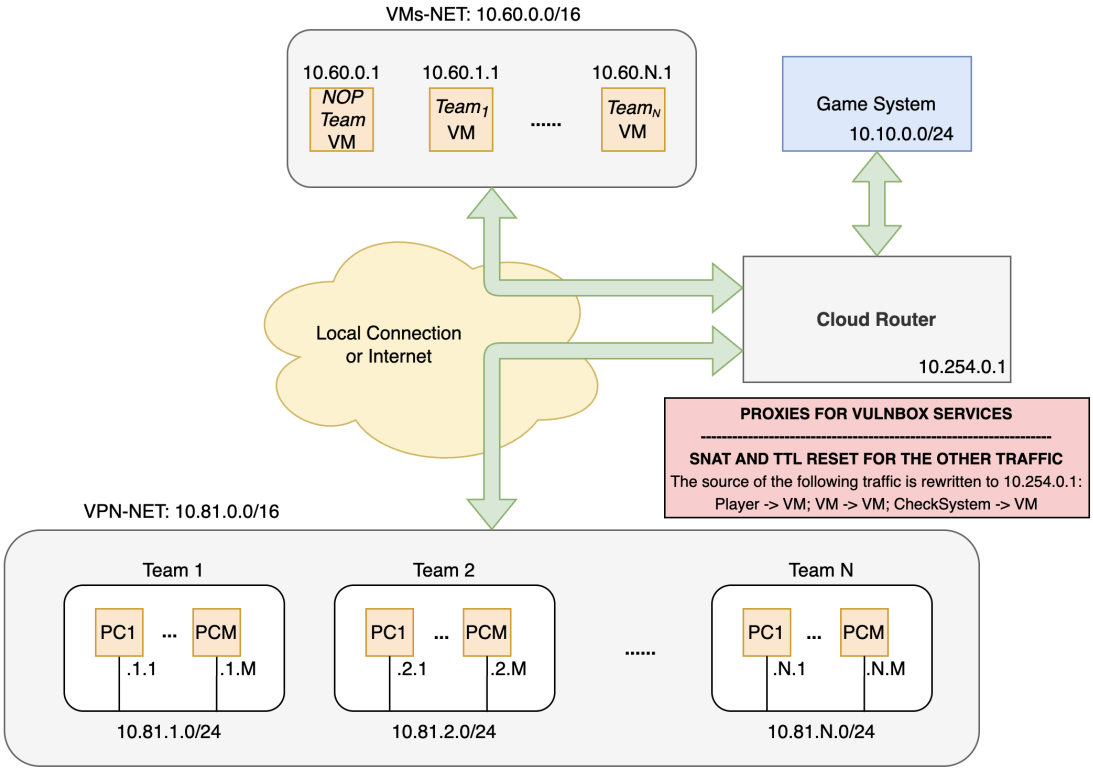

Figura 1.2: Modello della rete di gara di Cyberchallenge[9].

Al centro di tutto ciò troviamo il cloud router, che si occupa di gestire il traffico in base alle regole imposte dagli organizzatori e in base alla fase in cui si trova la competizione, ma che soprattutto si occupa di anonimizzare il traffico, manipolandolo tramite diverse tecniche tra le quali il Network Address Translation (NAT). Tramite il NAT infatti si nascondono gli indirizzi Internet Protocol (IP) delle macchine e dei componenti del team, mascherando nelle richieste la vera identità del mittente con quella del router stesso. Questo è un elemento di fondamentale importanza poiché altrimenti sarebbe davvero molto semplice diversificare le connessioni dei checker da quelle degli attaccanti, andando a perdere l'obiettivo che ha la competizione stessa, ovvero: individuare, sfruttare e difendersi dalle vulnerabilità.

Inoltre, si specifica che le reti dei singoli team sono isolate per tutta la competizione al fine di evitare possibili attacchi agli host dei partecipanti, che non sono dei target di interesse nella gara e per cui non è consentita l'esecuzione di attacchi mirati. Un simile comportamento potrebbe portare alla squalifica del team attaccante a seguito di segnalazioni e analisi del traffico da parte degli organizzatori.

1.2 Svolgimento della competizione

Una competizione A/D inizia nel momento in cui viene dato accesso alle macchine e alla rete di gara. Usualmente in questa fase c'è un periodo di tempo in cui la rete rimane non accessibile: i team avranno accesso unicamente alle loro macchine, mentre l'accesso ai servizi degli avversari verrà negato. In questo momento tuttavia è già possibile analizzare i servizi, avviare tutti i tool necessari all'avvio degli attacchi, all'analisi della rete, e alla difesa del server (che analizzeremo nel prossimo paragrafo). Finito questo periodo di tempo, la rete di gara viene aperta, e la competizione inizia ufficialmente.

Da ora il gameserver inizia a scandire i tick, i checker iniziano ad utilizzare i servizi e ad inserire le prime flag, pubblicando anche i primi Flag ID. I team iniziano a lavorare, tentando di difendersi dai primi attacchi, e a loro volta di attaccare rubando le prime flag agli avversari, tutto ciò mantenendo sempre i loro servizi attivi e funzionanti. Ad ogni tick, il gameserver assegna i nuovi punteggi, e i team possono osservare tramite questi dati l'evoluzione della gara stessa, comprendendo quali possono essere le azioni da compiere per massimizzare la crescita del loro punteggio. Inoltre, viene segnalato pubblicamente quali dei servizi di quale team non sono funzionanti, e perchè sono considerati down: cioè la motivazione per cui i checker hanno segnalato un malfunzionamento. La competizione termina ad un determinato orario di fine, dove la rete di gara viene chiusa, e il gameserver interrompe l'azione dei checker non accettando, inoltre, nuove submission di flag.

Quanto descritto in questo capitolo è possibile simularlo e sperimentarlo tramite il progetto CTFBox[10], nato come fork di Oasis dei TheRomanXpl0it[11] con cui è possibile simulare ma anche svolgere competizioni A/D di medio-piccole dimensioni in un ambiente realizzato unicamente tramite l'utilizzo di container, con funzionamento e regolamento simile a quello di CyberChallenge[9].

1.3 Strumenti utili durante la competizione

Data l'elevata complessità nello svolgimento di una competizione A/D, spesso diventa necessario l'utilizzo di tool specializzati che permettano di automatizzare diverse operazioni necessarie durante la gara. Ogni team può sviluppare e utilizzare anche tool che facilitano (o perfino automatizzino) l'attacco e la difesa dei servizi.
In generale i tool utilizzati più frequentemente sono quelli descritti di seguito.

[10] `https://github.com/domysh/ctfbox`

[11] `https://theromanxpl0.it/about.html`

[9] `https://ad.cyberchallenge.it/rules/`

1.3.1 Traffic Analyzer

I tool di analisi del traffico permettono di investigare il flusso di dati sulla macchina, in modo da riconoscere e visionare il traffico dei checker e degli attaccanti (anche se anonimizzato). Analizzare il traffico diventa cruciale dal momento in cui un nuovo attacco è attivo sulla rete, poiché permette ai team di analizzarlo, riconoscere la vulnerabilità sfruttata e intraprendere azioni quanto più immediate per la difesa e la replicazione dello stesso attacco. Normalmente per analizzare il traffico è possibile usare tool tradizionali quali Wireshark[12] o Suricata[13], che tuttavia risultano poco pratici nel loro utilizzo in ambito CTF A/D dove il traffico da analizzare è elevato e continuo. Esistono infatti, tool scritti appositamente per queste competizioni che permettono di analizzarlo in modo più efficiente e veloce, ma anche di riprodurre velocemente gli stessi payload di attacco, come ad esempio Caronte[14] o Tulip[15].

Ad esempio Caronte[14] avvantaggia particolarmente l'analisi del traffico grazie alla sua interfaccia grafica interattiva e di facile utilizzo, che permette di visualizzare i flussi di traffico TCP aggregati e decodificati in pochissimi click, permettendone anche il tagging tramite l'utilizzo di Regular Expression (regex) personalizzate, che permettono ad esempio di identificare una flag in uscita (quindi rubata o controllata dal checker) e le flag in entrata (quindi le nuove flag inserite dal gameserver). Ogni connessione è classificata tramite tag e colori personalizzabili in base alla porta di destinazione (e.g. HTTP: blu, SSH: verde), permettendo una mappatura visiva immediata dei servizi sotto attacco. Inoltre include una timeline interattiva, campionata al minuto per scorrere la porzione di traffico che si desidera analizzare. Si riporta il logo del progetto nella Figura 1.3, e un'immagine illustrativa dell'interfaccia nella Figura 1.4.

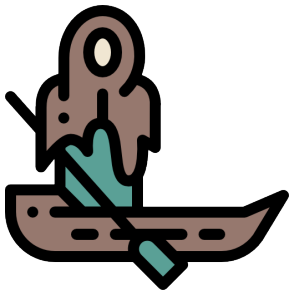

Figura 1.3: Logo di Caronte[14].

[12] https://www.wireshark.org/

[13] https://suricata.io/

[14] https://github.com/eciavatta/caronte

[15] https://github.com/OpenAttackDefenseTools/tulip

Figura 1.4: Interfaccia di Caronte[14].

1.3.2 Attacker & Submitter

Gli strumenti di attacco e submission automatizzati coordinano gli attacchi, assistono nella scrittura degli exploit e ne monitorano l'esecuzione, programmandola e parallelizzandola. Questi tool sono fondamentali per permettere all'attaccante di concentrarsi sulla progettazione del singolo attacco, anziché sulla gestione della sua esecuzione su tutte le macchine nemiche. Inoltre, grazie alla tracciabilità degli attacchi, è possibile identificare problemi durante l'esecuzione e monitorare quali team iniziano a difendere i propri servizi. Solitamente (ma non sempre) questi strumenti gestiscono anche la submission delle flag al gameserver in modo centralizzato, semplificando il rispetto delle regole organizzative che limitano frequenza e quantità per prevenire attacchi bruteforce o Denial of Service (DoS) all'infrastruttura.

Esempi di tali tool includono DestructiveFarm[16], che presenta tuttavia criticità nell'analisi dell'andamento degli attacchi, e ExploitFarm[17], sviluppato proprio per ovviare a queste limitazioni. A differenza del primo, il secondo offre un coordinamento più efficiente degli attacchi, un monitoraggio avanzato e una schedulazione semplificata, supportando attacchi distribuiti su più macchine con funzionalità di analisi in tempo reale.

La sua infrastruttura ricalca quella di DestructiveFarm[16]: un server centrale gestisce gli exploit e le submission, mentre client distribuiti eseguono gli attacchi e inviano le flag al server, bilanciando il carico computazionale derivante dalla replicazione degli exploit. Il sistema include un client terminale per l'esecuzione degli

attacchi e un'interfaccia web centralizzata per il monitoraggio e l'analisi. La piattaforma è orientata alla raccolta dati e alla reportistica, con funzionalità di versioning degli exploit, registrazione dettagliata di ogni esecuzione e analisi delle cause di fallimento per ottimizzare gli attacchi.

La dinamicità del sistema consente modifiche in *real-time*, propagate immediatamente all'intera infrastruttura, e notifiche automatiche per criticità o errori rilevati durante le operazioni. Si riporta il logo del progetto nella Figura 1.5, e un'immagine illustrativa dell'interfaccia nella Figura 1.6.

Figura 1.5: Logo di ExploitFarm.

Figura 1.6: Interfaccia di ExploitFarm.

1.3.3 Proxy e Firewall

Questi tool permettono di filtrare il traffico in modo da riconoscere e bloccare gli attacchi. Essi sono molto utili nella difesa, e permettono di bloccare gli attacchi

senza di fatto eseguire una modifica sui servizi stessi, spesso diretta conseguenza di malfunzionamenti per errori eseguiti durante questa fase. Questo modo di proteggere i servizi, tuttavia, risulta spesso incompleto e superabile, ma se ben progettato e correttamente utilizzato può essere un'ottima soluzione temporanea, se non una soluzione completa in alcuni casi specifici. Spesso però la soluzione più efficace rimane comunque quella di correggere la vulnerabilità. Strumenti di questo genere sono ad esempio CTF Proxy[18] o quello di cui tratta questo lavoro di tesi: Firegex[4]. Si riporta il logo del progetto nella Figura 1.7.

Figura 1.7: Logo di Firegex.

1.4 Esempio di un servizio vulnerabile: PCSS

Consideriamo come esempio un servizio vulnerabile presente nel simulatore di competizioni A/D CTFBox[10], denominato PCSS (Permanent Cat Storage Service). Il servizio è composto da un semplice file python e permette di memorizzare e leggere dei testi tramite una connessione TCP attraverso un'interfaccia Command Line Interface (CLI) molto semplice. Il servizio presenta un solo flag store, ma ha al suo interno due vulnerabilità, pertanto per renderlo sicuro è necessario correggerle entrambe.

La registrazione al servizio avviene in modo anonimo: selezionando la funzione di creazione del database, il servizio fornisce un id alfanumerico del database, e un token jwt di accesso con cui è possibile accedere al medesimo in seguito. Una volta registrati, o eseguito l'accesso tramite il token, è possibile elencare i file presenti nel database, leggerne il contenuto, e crearne nuovi. Nel Codice 1.3 si mostra l'interfaccia CLI del servizio.

[18] `https://github.com/ByteLeMani/ctf_proxy`

[4] `https://github.com/Pwnzer0tt1/firegex`

[10] `https://github.com/domysh/ctfbox`

```
PCSS - Permanent Cat Storage Service

          /\_/\
         ( o.o )
          > ^ <
      __/_______\__
     |             |
     |   WELCOME   |
     |_____________|

Meow! I'm a cat and I'm here to help you store your files.
A more secure, reliable and fun way to store your files.

--------------------------------------------------
1. Create a new database
2. Access an existing database
3. Exit

Insert your choice:
```

Codice 1.3: Interfaccia CLI di PCSS (da CTFBox[10]).

A livello tecnico il servizio crea una cartella per ogni database, e all'interno di essa crea i file richiesti. Il gameserver pubblica come Flag ID l'id del database in cui si trova la flag, e il nome del file in cui è memorizzata.

Come è possibile notare, i Flag ID non sono sufficienti per accedere alla flag, ma sono un'informazione utile per gli attaccanti per individuare più facilmente la flag all'interno del servizio, e inoltre ci è di aiuto per comprendere dove il servizio memorizza le flag, in questo caso in un file nel database.

La prima vulnerabilità che possiamo individuare è una vulnerabilità di Local File Inclusion (LFI), causata dall'implementazione della funzione nel Codice 1.4.

```python
def read_file():
    file = input("Please insert the file name: ").strip()
    print("File content:")
    subprocess.run(["cat", f"./data/{ctx.loggined_db}/{file}"])
    print("")
```

Codice 1.4: Funzione vulnerabile a Local File Inclusion nel servizio PCSS (da CTFBox[10]).

Inserendo semplicemente `'../<database_id>/<file_name>'` come nome del file, è possibile accedere a file esterni al database, e quindi anche alla flag.

La seconda vulnerabilità invece è una vulnerabilità di natura crittografica causata dall'uso di una vecchia versione della libreria `jwt` (versione 0.5.4), che permette di inserire nel token come algoritmo di firma `none` e quindi di *bypassare* la verifica della firma, permettendo di generare un token malevolo per accedere al database. Questo

è possibile poiché la prima parte del token JSON Web Token (JWT) è in chiaro, e la sua autenticità non è garantita, per cui è usualmente buona pratica specificare lato server quali algoritmi di firma sono supportati, escludendo attacchi di questo tipo. Questa vulnerabilità è la medesima riscontrata in un famoso framework di autenticazione JWT in NodeJS e classificata come CVE-2022-23540[19].

È pertanto possibile costruire un token JWT valido per accedere al database, e quindi alla flag, senza conoscere il segreto di firma come mostrato nel Codice 1.5.

```
def attack(team_ip: str, flag_id):
    #The bugged jwt is encoded manually but you could also use a jwt lib to
    ↪  make it
    first_part = base64.b64encode(json.dumps({"alg": "none", "typ":
    ↪  "JWT"}).encode()).decode().replace("=", "")
    data_part = base64.b64encode(json.dumps({'db':
    ↪  flag_id['db_name']}).encode()).decode().replace("=", "")
    jwt = f"{first_part}.{data_part}."
    conn = CatStorage(team_ip)
    conn.login(jwt)
    result = conn.read_file(flag_id["filename"])
    conn.close()
    return result
```

Codice 1.5: Funzione per eseguire un attacco tramite un token JWT malevolo su PCSS (da CTFBox[10]).

La difesa di questo servizio è piuttosto semplice, ma potrebbe risultare anche facilmente soggetta ad errori che potrebbero comprometterne il funzionamento: ciò potrebbe essere causato dall'aggiornamento della libreria `jwt` che se soggetta a cambiamenti alle sue API nelle versioni successive, potrebbe portare a malfunzionamenti imprevisti del servizio stesso, e quindi necessitare di ulteriori modifiche al servizio.

In alternativa potremmo utilizzare sistemi come firegex[4] che tramite la sua nuova funzione `nfproxy` può eseguire la decodifica del token JWT per verificare che l'algoritmo nel token sia quello previsto dal servizio, interrompendo i tentativi di accesso malevoli. Inoltre, filtrando anche i doppi punti nel traffico TCP in ingresso, possiamo proteggere il servizio anche dalla vulnerabilità di LFI.

[19] https://nvd.nist.gov/vuln/detail/cve-2022-23540

[4] https://github.com/pwnzer0tt1/firegex

Capitolo 2

Firegex

Firegex è un SDFW innovativo e multifunzionale, ideato per essere avviato con 0 configurazioni in modo rapido e semplice, che agisce principalmente a livello applicativo per permettere un'analisi del traffico specifica e dettagliata per ogni servizio. Esistono al suo interno diversi moduli che permettono la creazione di filtri di diversa natura, in modo semplice, flessibile e quanto meno invasivo ed impattante sulle prestazioni della macchina, mantenendo sempre la massima priorità sulla disponibilità e la corretta funzionalità del servizio stesso.

Il contesto per cui è stato pensato è quello delle competizioni CTF di tipo A/D, dove vi sono particolari esigenze (descritte nel Capitolo 1), che Firegex cerca di soddisfare, semplificandone il suo utilizzo, e soprattutto fornendo soluzioni adattabili ai requisiti spesso fortemente dinamici dei servizi in gara. Nella Figura 2.1 si mostra l'interfaccia grafica di Firegex.

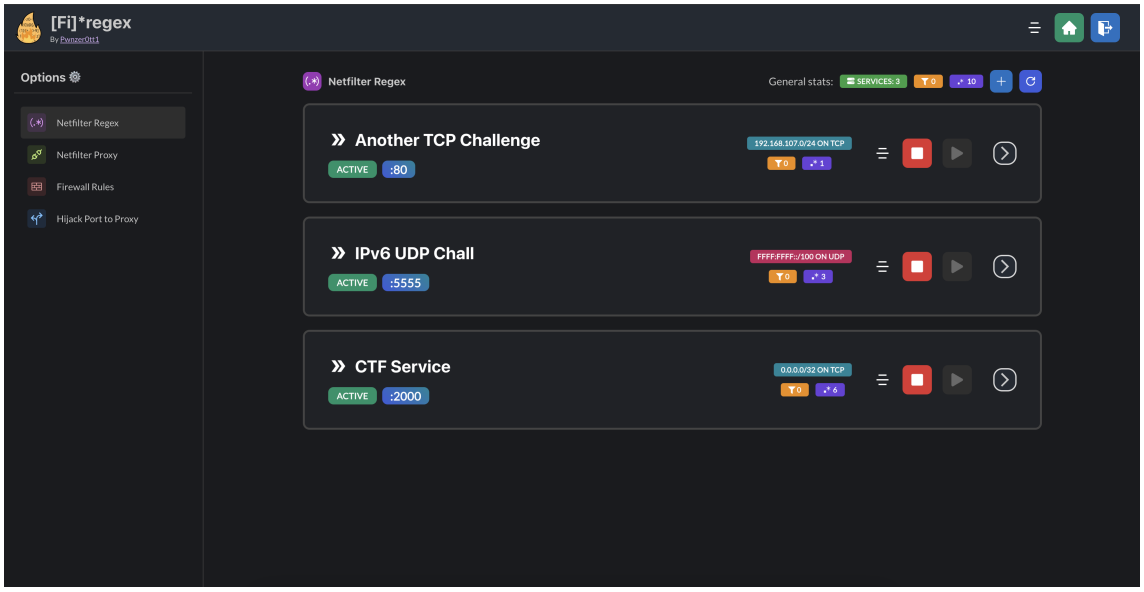

Figura 2.1: Interfaccia grafica di Firegex.

2.1 Perché nasce Firegex?

Firegex (come detto in precedenza) nasce dal gruppo che ha affrontato la gara nazionale di Cyberchallenge[6] del 2022 al Politecnico di Bari[5], come un normale proxy con filtri basati sulle regex, che col tempo si è evoluto aggiungendo nuove funzionalità sempre più efficienti, sicure e funzionali alla competizione stessa.

La sua necessità è nata proprio dalla mancanza di un sistema di difesa che fosse immediato da utilizzare, che non richiedesse configurazioni complesse per essere avviato, e che permettesse in maniera semplice di costruire dei filtri prontamente attivi, funzionanti e stabili.

Durante la sua progettazione, ci si è sempre più focalizzati su alcuni punti chiave che lo rendono unico rispetto ad altre soluzioni presenti:

- **Trasparenza totale:** La creazione di un proxy comporta una serie di cambiamenti sulla configurazione delle porte di rete esposte dai servizi, l'avvio e la gestione del processo del proxy stesso, e utilizzo di meccanismi di *fallback* per ovviare ad eventuali problematiche. Queste complicanze erano solo parzialmente gestite inizialmente da firegex, ma con le nuove implementazioni è possibile applicare regole sul traffico senza cambiamenti nelle configurazione, grazie a particolari funzionalità utilizzate, disponibili nel kernel Linux, nascondendone però tutta la complessità gestita internamente per il loro corretto funzionamento;
- **Integrazione di tecnologie multiple:** Come già detto, Firegex utilizzava come unico meccanismo di filtraggio le regex, che venivano applicate sui pacchetti di rete per individuare eventuali attacchi. Negli anni si è notato come questo approccio spesso potesse essere limitante, per cui sono stati implementati nuovi meccanismi e tecnologie di filtraggio aumentando la precisione con cui è possibile analizzare il traffico. La seguente tesi tratterà di una sua nuova funzionalità particolarmente flessibile denominata `nfproxy`;
- **Affidabilità e disponibilità:** In ambienti dove la continuità del servizio è critica, è fondamentale che il sistema di sicurezza non diventi un collo di bottiglia. Firegex è stato progettato per garantire un'elevata disponibilità, avendo meccanismi di *fallback* applicati in caso di errori critici, e offrendo soluzioni rapide per il ripristino della configurazione in caso di errori e per il reset immediato dei filtri applicati;
- **Facilità di utilizzo:** Nelle A/D il tempo è una risorsa fondamentale: gli strumenti a disposizione per la gara non devono richiedere eccessivi tempi di configurazione e devono facilitare la risoluzione di problemi semplificando le

[6] `https://cyberchallenge.it/`

[5] `https://www.poliba.it`

eventuali fasi di troubleshooting, che possibilmente andrebbero evitate. L'obiettivo infatti è quello di focalizzare l'attenzione sull'individuazione di vulnerabilità, che rappresenta un necessità chiave nella competizione. Firegex è stato progettato per essere quanto meno dispersivo possibile, avere configurazioni semplici e guidate, un'interfaccia intuitiva, e gestire autonomamente tutte le problematiche legate alla gestione dei filtri e alla loro esecuzione.

2.2 Confronto con altre soluzioni

Firegex si distingue rispetto ad altre soluzioni presenti, come CTF proxy[18], Nginx[20], Suricata[13] o i Next Generation Firewall (NGFW) per diverse motivazioni mostrate nella Tabella 2.1.

Firewall	0 Config Run	Easy use	Easy Filter Add	Flexible
Firegex	✅	✅	✅	✅
Nginx	⚠	⚠	⚠	⚠
CTF proxy	❌	⚠	⚠	✅
NGFW	❌	❌	⚠	⚠
Suricata	❌	⚠	⚠	⚠

Tabella 2.1: Confronto tra le soluzioni per il firewall.

- **Nginx** è una soluzione molto flessibile ed affidabile, ma richiede configurazioni complesse e non permette di applicare filtri in maniera immediata e trasparente, ha necessità di cambiare porte e non supporta nativamente filtri più avanzati come quello trattato in questo lavoro di tesi. Non ha un'interfaccia grafica per la gestione delle regole;
- **CTF proxy** è una soluzione fortemente flessibile, ma richiede la creazione di configurazioni e allo stesso modo necessita di cambiare porte ai servizi originali, inoltre a suo supporto non esiste un'interfaccia semplice da usare;

[18] https://github.com/ByteLeMani/ctf_proxy

[20] https://nginx.org/

[13] https://suricata.io/

- I **NGFW** offrono soluzioni estremamente affidabili, ben riconosciute nel settore, ma richiedono spesso configurazioni complesse e sono poco intuitivi da utilizzare. Pertanto il loro utilizzo risulta poco pratico per competizioni CTF, nonostante rimangano le principali soluzioni applicate al di fuori di questo contesto;
- **Suricata** è una soluzione molto affidabile che offre un grado di flessibilità intermedio, ma pecca della mancanza di un'interfaccia semplice per il suo utilizzo, e richiede configurazioni per essere avviato.

2.3 Integrazione con il networking su Linux

In questa parte si descrive come Firegex si integra con il networking su Linux, in particolare con il framework Netfilter e le sue tabelle, che sono alla base della gestione del traffico di rete nel kernel Linux. Il sistema target analizzato è basato su Linux dato il suo ampio utilizzo nelle competizioni CTF ma anche in contesti di produzione, dove la flessibilità e le prestazioni sono fondamentali.

Quando un pacchetto di rete raggiunge l'interfaccia di una macchina Linux, viene avviato un processo di elaborazione articolato che attraversa diversi stadi nel kernel. Il pacchetto viene inizialmente ricevuto dall'hardware di rete e passato al driver della scheda, che lo trasferisce al sottosistema di rete del kernel attraverso interrupt o polling. Una volta nel kernel, il pacchetto attraversa il networking stack di Linux, passando per diverse fasi di elaborazione sequenziali.

Durante questo percorso, il framework Netfilter inserisce punti di intercettazione chiamati **hook** in momenti strategici del flusso di elaborazione. Questi hook, visibili nella Figura 2.2, permettono di eseguire operazioni personalizzate sui pacchetti in transito senza modificare il codice del kernel. I principali hook sono:

- **PREROUTING**: Eseguito immediatamente dopo la ricezione del pacchetto, prima delle decisioni di routing;
- **INPUT**: Attivato per pacchetti destinati al sistema locale;
- **FORWARD**: Gestisce pacchetti in transito verso altre destinazioni;
- **OUTPUT**: Elabora pacchetti generati localmente;
- **POSTROUTING**: Ultimo hook prima della trasmissione del pacchetto.

Ogni hook supporta l'esecuzione di regole con diverse **priorità** (prio), valori numerici che determinano l'ordine di esecuzione delle operazioni. Priorità negative vengono eseguite prima di quelle positive, permettendo un controllo granulare sulla sequenza di elaborazione. Ad esempio, operazioni di deframmentazione avvengono tipicamente a priorità -400, mentre il modulo `conntrack` opera a priorità -200.

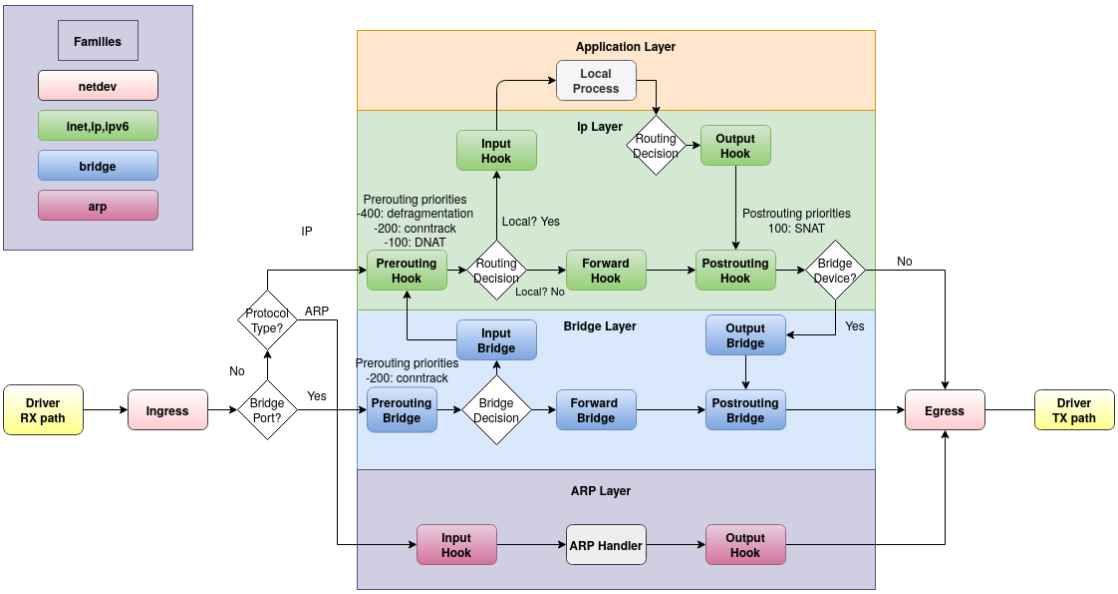

Figura 2.2: Grafico degli hooks di Netfilter Tables.

Il framework Netfilter Tables (nftables) rappresenta l'evoluzione moderna del sistema di filtraggio Linux, sostituendo il precedente sistema xtables utilizzato da iptables. Mentre iptables continua a essere ampiamente utilizzato per compatibilità, oggi impiega nftables come backend, beneficiando delle sue prestazioni superiori e della sintassi semplificata. Questo processo di transizione porterà gradualmente all'eliminazione completa di xtables in favore dell'architettura più efficiente di nftables.

Tra i moduli che operano negli hook di Netfilter, particolare rilevanza assume `conntrack`, responsabile del tracciamento delle connessioni. Questo modulo mantiene lo stato delle sessioni di rete, distinguendo tra nuove connessioni, traffico correlato a connessioni esistenti e pacchetti di risposta. Le informazioni gestite da `conntrack` sono fondamentali per l'implementazione di firewall stateful e per le operazioni di NAT.

2.3.1 Netfilter Tables

nftables rappresenta il framework moderno per il filtraggio dei pacchetti e la gestione del traffico nelle distribuzioni Linux. Questo sistema permette di definire regole che vengono eseguite nei vari hook del percorso di elaborazione dei pacchetti, offrendo un controllo granulare e prestazioni ottimizzate.

In Firegex, nftables costituisce il meccanismo fondamentale per intercettare e gestire il traffico di rete attraverso diverse modalità operative:

- **Applicare regole di sicurezza:** Le regole di firewall tradizionali vengono implementate direttamente tramite nftables, sfruttando le sue capacità native

di filtraggio per bloccare, accettare o modificare il traffico in base a criteri specifici (indirizzi IP, porte, protocolli);

- **Reindirizzamento del traffico:** La funzionalità `Netfilter Port Hijack (porthijack)` utilizza le regole di NAT di nftables per deviare trasparentemente il traffico da una porta a un'altra, permettendo l'inserimento di proxy intermedi senza modifiche alla configurazione dei servizi;
- **Integrazione con Netfilter Queue (nfqueue):** nftables fornisce la base per l'attivazione del modulo nfqueue, definendo quando e quali pacchetti devono essere trasferiti dallo spazio kernel allo spazio utente per elaborazioni avanzate.

2.3.2 Netfilter Queue Module

Il modulo nfqueue rappresenta un'estensione specializzata del framework Netfilter che permette di trasferire pacchetti dallo spazio kernel allo spazio utente per elaborazioni complesse. Questo modulo opera come target delle regole nftables, attivandosi quando una regola specifica il verdict `queue` su un pacchetto che attraversa uno degli hook di Netfilter.

Il funzionamento di nfqueue segue un processo ben definito:

1. Una regola nftables intercetta un pacchetto in uno degli hook del percorso di elaborazione;
2. Se la regola specifica l'azione `queue`, il pacchetto viene trasferito a una coda numerata gestita da nfqueue;
3. Un'applicazione userspace si registra per ricevere pacchetti da quella specifica coda;
4. L'applicazione elabora il pacchetto e restituisce un verdict (ACCEPT, DROP, etc.), ed eventualmente modifica il pacchetto stesso;
5. Il kernel applica il verdict ricevuto, continuando o interrompendo l'elaborazione del pacchetto.

Questa architettura offre vantaggi significativi per Firegex:

- **Accesso completo ai pacchetti:** L'applicazione userspace riceve l'intero pacchetto, inclusi header di rete e trasporto, permettendo analisi approfondite e modifiche precise non possibili con proxy tradizionali;
- **Trasparenza totale:** L'intercettazione avviene a livello kernel senza richiedere modifiche alla configurazione dei servizi applicativi, che rimangono completamente ignari del processo di filtraggio;
- **Flessibilità nell'elaborazione:** Lo spazio utente permette l'implementazione di logiche complesse (parsing di protocolli, analisi con machine learning, integrazione con database) mantenendo la stabilità del kernel;

- **Controllo granulare del flusso:** La possibilità di modificare pacchetti e restituire verdict personalizzati offre un controllo fine sul comportamento della rete.

I moduli `Netfilter Regex Filter (nfregex)` e `nfproxy` di Firegex sfruttano intensivamente questa architettura, registrandosi come listener su code nfqueue specifiche e implementando le rispettive logiche di filtraggio in spazio utente.

2.4 Architettura

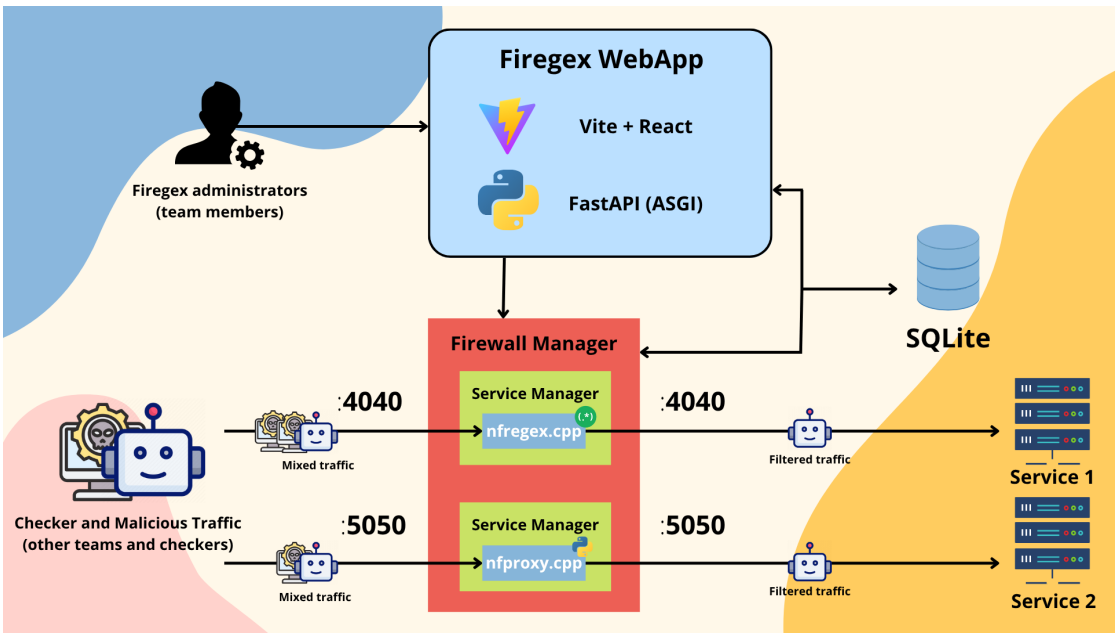

Figura 2.3: Schema generale dell'architettura di Firegex.

Firegex, avendo un interfaccia web, è composto da un frontend scritto in react[21] (usando vite[22]) e un backend sviluppato con fastapi[23]. Il backend utilizza inoltre dei moduli scritti in C++ per le funzionalità più critiche, come per lo scambio dei pacchetti lato kernel e l'elaborazione in tempo reale delle espressioni regolari. Inoltre per funzionare, fa intenso utilizzo delle funzionalità di nftables[24], e di nfqueue[25].

[21] `https://react.dev/`

[22] `https://vite.dev/`

[23] `https://fastapi.tiangolo.com/`

[24] `https://netfilter.org/projects/nftables/`

[25] `https://netfilter.org/projects/libnetfilter_queue/`

L'intera infrastruttura è avviata tramite docker[26] che ne permette un avvio immediato e senza configurare parametri: docker è usualmente lo strumento principale per l'avvio di servizi in competizioni CTF, pertanto nella maggior parte dei casi è già presente e configurato correttamente. In Figura 2.3 è mostrato uno schema generale dell'architettura di Firegex.

2.4.1 Frontend e Backend

- **Frontend (React):** L'interfaccia utente risulta fortemente interattiva e intuitiva nelle fasi di creazione dei filtri, nel loro monitoraggio e la gestione degli stessi. L'interfaccia è parte fondamentale di Firegex, poiché consente di soddisfare uno dei punti chiave del progetto: la semplicità di utilizzo;
- **Backend (FastAPI + C++):** Il backend è realizzato utilizzando FastAPI, un framework python moderno e ad alte prestazioni per la creazione di API HTTP, integrato con moduli critici scritti in C++. Questa combinazione consente di gestire le richieste in maniera estremamente veloce ed affidabile, garantendo al contempo, l'esecuzione delle operazioni critiche a livello di rete con le performance di un linguaggio a basso livello. Il backend inoltre svolge anche un ruolo attivo nell'applicazione delle regole di rete tramite il modulo ufficiale di libnftables[27].

2.4.2 Moduli C++

Le parti critiche di networking gestite da Firegex sono implementate in C++. I moduli di firegex che lo prevedono infatti, hanno un eseguibile compilato, responsabile del filtraggio, che si occupa di intercettare i pacchetti dal kernel Linux e di implementare la logica prevista dalla funzionalità per cui è stato realizzato. I moduli che attualmente fanno utilizzo di questi eseguibili sono `nfregex` e `nfproxy`. Essi fanno utilizzo di librerie aventi bassi tempi di elaborazione (permettendo alte prestazioni in rete) come libtins[28], libmnl[29], vectorscan[30] (specificatamente per `nfregex`), e le C API di python[31] (specificatamente per `nfproxy`). La scelta del linguaggio C++ ha permesso di perseguire i seguenti obiettivi:

[26] https://www.docker.com/

[27] https://git.netfilter.org/nftables/tree/py

[28] https://libtins.github.io/

[29] https://netfilter.org/projects/libmnl/

[30] https://vectorcamp.gr/project/vectorscan/

[31] https://docs.python.org/3/c-api/

- **Prestazioni elevate**: Grazie all'efficienza del linguaggio C++ e alla possibilità di gestire le interazioni a basso livello con il sistema operativo, il filtraggio del traffico avviene con latenze estremamente ridotte;
- **Parallelizzazione**: I filtri che richiedono elaborazione userspace facendo uso di nfqueue, sono gestiti in un sistema di elaborazione multi-thread, garantendo una gestione ottimale del traffico in situazioni di carico elevato e con un numero di connessioni elevate;
- **Integrazione con librerie di rete**: L'utilizzo di librerie specializzate come libtins e libmnl (anch'esse scritte in C++), consente di implementare funzionalità avanzate mantenendo affidabilità su come vengono elaborati i pacchetti in rete, facendolo con un costo computazionale basso.

2.5 Funzionalità

All'interno di Firegex sono presenti attualmente quattro moduli principali, ognuno con delle funzionalità specifiche che permettono di applicare filtri con tecnologie e funzionalità differenti. I moduli sono `nfregex`, `firewall`, `porthijack` e `nfproxy`. All'interno di questi moduli firegex possiede una documentazione interna alla piattaforma stessa che permette di comprendere come utilizzare le funzionalità offerte, e come configurare i filtri in maniera corretta.

Firegex è avviabile tramite un unico comando mostrato in Figura 2.1, supponendo che docker[26] sia stato già installato nel sistema.

```
sh <(curl -sLf https://pwnzer0tt1.it/firegex.sh)
```

Codice 2.1: One-Command per l'esecuzione di Firegex.

2.5.1 Modulo nfregex

Il modulo `nfregex` (pensato originariamente come unico e principale) possiede funzionalità per la creazione di filtri basati su espressioni regolari. Utilizzando nfqueue in combinazione con nftables, questo modulo intercetta le richieste di rete direttamente a livello kernel. La funzionalità permette l'utilizzo di regex Perl Compatible Regular Expressions (PCRE)[32] *compliant* tramite l'utilizzo di vectorscan[30] (fork di

[26] https://www.docker.com/

[32] https://www.pcre.org/

[30] https://vectorcamp.gr/project/vectorscan/

hyperscan[33] compatibile con arm64) per garantire prestazioni elevate e una corretta elaborazione delle regole. La funzionalità supporta Internet Protocol version 4 (IPv4) e Internet Protocol version 6 (IPv6), e protocolli di livello superiore come TCP e User Datagram Protocol (UDP). Per i pacchetti TCP, la regex viene applicata all'intero flusso ricostruito e ordinato, tramite le stream regex disponibili in vectorscan.

2.5.2 Modulo firewall

La funzionalità `firewall` di Firegex offre un controllo completo sul traffico di rete, tramite regole simili a quelle offerte da strumenti che agiscono a livello di rete e di trasporto come Uncomplicated Firewall (UFW) o iptables. Dispone quindi di funzionalità basilari e risulta utile per applicare questa tipologia di regole evitando ulteriori tool di filtering che potrebbero entrare in conflitto con firegex.

2.5.3 Modulo porthijack

Il modulo `porthijack` introduce la capacità di reindirizzare il traffico destinato ad una determinata porta verso un'altra. Questa funzionalità è particolarmente utile in scenari dove si ha il bisogno di costruire un proxy personalizzato, ma si vuole evitare la riconfigurazione del servizio di base. `Porthijack` infatti consente di impostare un redirect del traffico dall'interfaccia su cui il servizio viene esposto nella rete di gara, dirottandolo per un proxy in *localhost*, che poi tramite la stessa interfaccia di *loopback* inoltrerà il traffico al servizio originale.

2.5.4 Modulo nfproxy

`nfproxy`, il modulo trattato in questo lavoro di tesi, sfrutta le funzionalità di `nfqueue` per l'intercettazione del traffico, simulando lato utilizzatore il comportamento di un tradizionale proxy python. Le caratteristiche principali del modulo sono:

- **Sviluppo in Python:** Permette agli sviluppatori di scrivere e integrare filtri personalizzati in linguaggio Python, offrendo un altissimo livello di flessibilità per l'analisi del traffico;
- **Parsing avanzato dei protocolli:** Tramite funzionalità predefinite è possibile eseguire la decodifica di protocolli come HTTP, in modo immediato e già disponibile all'utilizzo, senza che queste funzionalità debbano essere implementate manualmente;

[33] `https://www.usenix.org/system/files/nsdi19-wang-xiang.pdf`

- **Test e validazione:** I filtri possono essere provati in anticipo tramite il comando fgex, rendendo possibile la prova del filtro prima si applicarlo al traffico;
- **Interfaccia grafica:** L'interfaccia di dettaglio del servizio permette di visualizzare in tempo reale i log e le eventuali eccezioni che si verificano durante l'esecuzione del filtro.

Il seguente modulo è stato introdotto in vista di scenari complessi dove gli elementi da individuare per il corretto filtraggio non sono facilmente individuabili tramite regex. In Figura 2.4 si mostra l'interfaccia grafica realizzata per il modulo `nfproxy`.

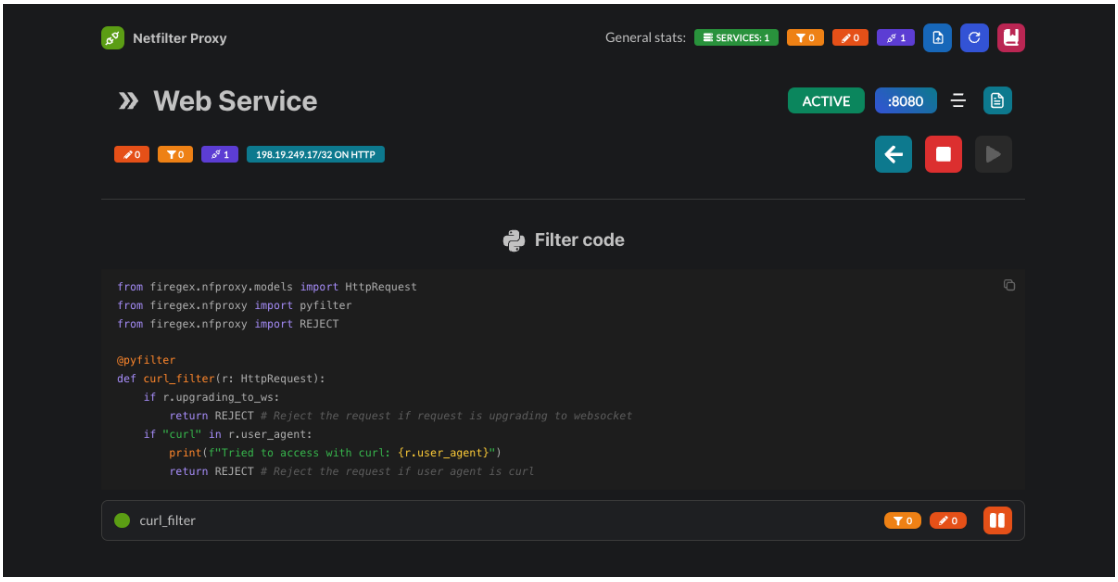

Figura 2.4: Interfaccia di dettaglio del servizio nel modulo nfproxy.

Nei prossimi capitoli verrà trattato in dettaglio `nfproxy` con le sue funzionalità, le problematiche affrontate e le soluzioni adottate.

Capitolo 3

Progettazione e sviluppo del modulo nfproxy

Il regex filtering è un modello di filtraggio che garantisce ottime prestazioni (grazie all'elaborazione tramite linguaggi e librerie ottimizzati), ma presenta forti limitazioni nella flessibilità di analisi del traffico. Questa rigidità è particolarmente rilevante nei CTF, dove la maggior parte dei servizi utilizza protocolli come HTTP, che spesso includono contenuti compressi o codificati (e.g., Base64, gzip), difficilmente analizzabili tramite regex. A discapito delle prestazioni, `nfproxy` introduce il vantaggio di scrivere filtri in Python[34], linguaggio semplice e rapido per lo scripting offrendo:

- Massima libertà nell'implementazione delle logiche di filtraggio;
- Prestazioni che, nonostante l'utilizzo di codice interpretato, restano elevate anche grazie a scelte architetturali mirate;
- Supporto nativo agli algoritmi di compressione e codifica più comuni, mediante il *parsing* del protocollo applicativo.

`nfproxy` non è un proxy tradizionale ma un modulo basato su nfqueue[25], che permette un controllo sul traffico paragonabile ad un proxy classico, ma con accesso diretto ai pacchetti a livello di rete (layer 3/4) e con un'intercettazione del traffico invisibile all'applicativo. Va riportata l'esistenza di limitazioni nella modifica dei pacchetti, che tuttavia non rappresenta un caso d'uso tipico della difesa del servizio.

Il sistema combina C++ per una gestione efficiente dei pacchetti e Python per eseguire il *parsing* dei protocolli applicativi. Questo approccio ibrido permette di sfruttare al meglio le caratteristiche di entrambi i linguaggi, garantendo prestazioni elevate e flessibilità nello sviluppo dei filtri.

---

[34] `https://python.org/`

[25] `https://netfilter.org/projects/libnetfilter_queue/`

I protocolli attualmente supportati includono:

- HTTP/1.1[35] (con gestione compressione/encoding del body);
- Websocket[36] (con decodifica dei frame e supporto alle estensioni);
- TCP standard[37] (con ricostruzione dello stream).

3.1 Scelte progettuali

Di seguito gli obiettivi fondamentali del modulo:

- **Integrazione con Python**: Fornire un'interfaccia in Python per implementare le regole di filtraggio, con funzioni ottimizzate e sintassi intuitiva;
- **Analisi avanzata di HTTP/1.1 e WebSocket**: Interpretazione nativa dei pacchetti applicativi, con estrazione automatica dei dati decodificati/decompressi. Supporto integrato per: gzip[38], deflate[39], brotli[40], zstd[41], e l'estensione WebSocket permessage-deflate[42];
- **Ricostruzione dello stream TCP**: Accesso al payload ordinato a livello di trasporto, indipendentemente dalla frammentazione;
- **Manipolazione degli header**: Lettura e modifica diretta dei campi negli header di rete (IP) e trasporto (TCP);
- **Compatibilità dual-stack**: Elaborazione sia di pacchetti IPv4[43] che IPv6[44];
- **Allocazione e deallocazione di memoria ottimizzata**: Gestione automatica dei buffer per evitare memory leak e sovraccarichi, non delegandone la gestione al programmatore, ma fornendo opzioni di configurazione opzionali per configurare il modo in cui vengono gestiti;

[35] RFC2616, Hypertext Transfer Protocol – HTTP/1.1

[36] RFC6455, The WebSocket Protocol

[37] RFC9293, Transmission Control Protocol (TCP)

[38] RFC1952, GZIP file format specification version 4.3

[39] RFC1951, DEFLATE Compressed Data Format Specification version 1.3

[40] RFC7932, Brotli Compressed Data Format

[41] RFC8478, Zstandard Compression and the application/zstd Media Type

[42] RFC7692, Compression Extensions for WebSocket

[43] RFC791, Internet Protocol

[44] RFC2460, Internet Protocol, Version 6 (IPv6) Specification

- **Diagnostica avanzata**: Notifica strutturata degli errori (codici, descrizioni, contesto) con log dettagliati per l'analisi forniti in tempo reale;
- **Modifica dinamica delle regole**: Attivazione/disattivazione istantanea dei filtri (anche solo di alcuni specifici) senza interrompere il servizio;
- **Elaborazione parallela**: Distribuzione del carico su multipli core della Central Processing Unit (CPU) per garantire scalabilità lineare;
- **Validazione dei filtri tramite simulazione**: Verifica delle regole in un ambiente simulato per debugging da poter utilizzare in fase di sviluppo e verifica dei filtri.

3.2 Architettura

Il modulo `nfproxy` viene avviato dal backend di Firegex attraverso un processo articolato in tre fasi principali.

In primo luogo, il backend configura le regole di nftables[24] mediante chiamate alle sue API JavaScript Object Notation (JSON). Successivamente, viene inizializzato il componente C++ responsabile della logica centrale, che include la gestione del multi-threading, l'elaborazione a basso livello dei pacchetti e la ricostruzione degli stream TCP. Nella fase finale, vengono caricati e applicati dinamicamente i filtri Python definiti dall'utente. L'architettura completa del modulo è rappresentata nella Figura 3.1, dove si evince l'interazione tra i tre livelli (configurazione, core C++ e strato Python).

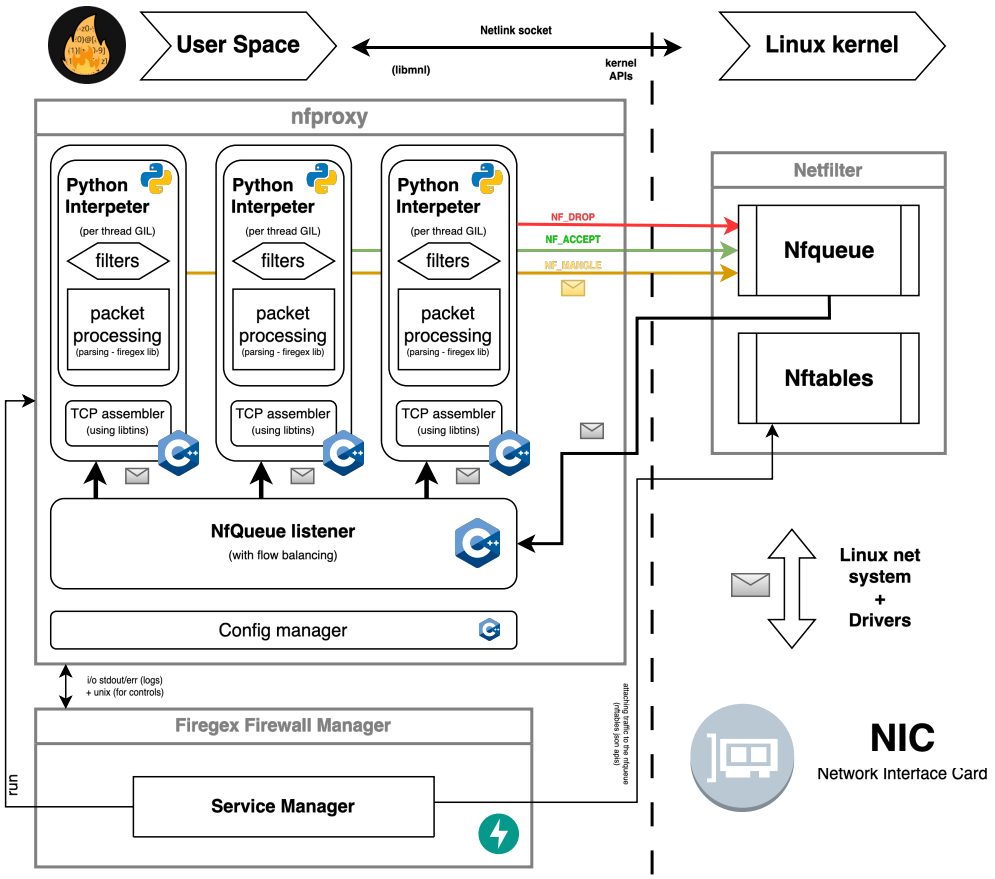

Figura 3.1: Architettura del modulo nfqueue.

[24] `https://netfilter.org/projects/nftables/`

3.3 Elaborazione parallelizzata dei pacchetti

Uno dei principali ostacoli progettuali ha riguardato la parallelizzazione dei processi di elaborazione, legata a tre fattori critici: il funzionamento intrinseco di nfqueue, alcune peculiarità del linguaggio Python e l'integrazione ibrida C++/Python.

Tre obiettivi hanno guidato l'implementazione della parallelizzazione:

- **Isolamento dei dati tra processi**: Prevenire conflitti interprocesso per evitare corruzione dati, crash imprevisti e comportamenti anomali;
- **Minimizzazione dei lock**: Limitare l'uso di meccanismi di blocco (pur necessari per risorse condivise) per prevenire deadlock e un degrado delle prestazioni;
- **Ottimizzazione della memoria**: Eliminare copie ridondanti dei pacchetti e processi superflui, massimizzando throughput e riducendo la latenza.

La soluzione adottata segue il pattern *producer-consumer*. Un processo produttore unico, responsabile del binding alla nfqueue, riceve i pacchetti dal kernel e li distribuisce a multiple code First In First Out (FIFO). I consumatori, associati ciascuno ad una coda dedicata, gestiscono in parallelo:

- Ordinamento dei pacchetti TCP;
- Applicazione dei filtri Python;
- Invio dei verdicts[45] al kernel.

L'unica sezione critica risiede nell'accesso concorrente alle code, protetto da un sistema di lock basato su std::mutex[46] e std::condition_variable[47]. Questa scelta, preferita alle unix pipe[48] per motivi prestazionali, implementa inoltre un meccanismo di backpressure che blocca il produttore in caso di code sature, prevenendo un consumo eccessivo di memoria.

L'implementazione si basa su quella pubblicata da Arif Jaffer[49] per le code bloccanti.

3.3.1 A Per-Interpreter Global Interpreter Lock

Una problematica riguardo la parallelizzazione riguarda una caratteristica del linguaggio Python: il Global Interpreter Lock (GIL)[50]. Questo meccanismo limita

[45] `https://netfilter.org/projects/libnetfilter_queue/doxygen/html/group__nfq__verd.html`

[46] `https://en.cppreference.com/w/cpp/thread/mutex`

[47] `https://en.cppreference.com/w/cpp/thread/condition_variable`

[48] `https://man7.org/linux/man-pages/man2/pipe.2.html`

[49] `https://www.bit-byter.com/blog/files/blocking-q-cpp.html`

[50] *Understanding the python gil*

l'utilizzo dei thread consentendo l'esecuzione di un solo thread alla volta, necessario per prevenire conflitti nella gestione interna degli oggetti Python (come per il *reference counting*) che non è implementata in modo *thread-safe*.

Si osserva inoltre come, in questo specifico contesto, Python gestisca operazioni *CPU-bound*, accentuando ulteriormente gli svantaggi del GIL. In Figura 3.2 è possibile visualizzare uno schema estratto da *Understanding the python gil*[50] che descrive il funzionamento del Python GIL

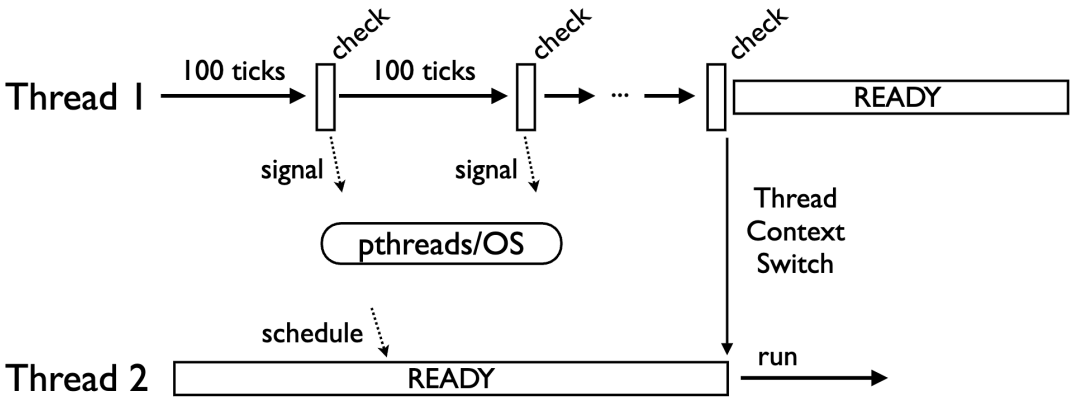

Figura 3.2: Python Global Interpreter Lock[50].

Una soluzione alternativa potrebbe essere l'uso di processi multipli (con GIL separati per processo), approccio che tuttavia introdurrebbe overhead dovuti alla comunicazione inter-processo e alla duplicazione della memoria.

La soluzione adottata sfrutta il Python Enhancement Proposal (PEP) 684[51] introdotto in Python 3.12, che permette l'esecuzione di interpreti Python indipendenti nello stesso processo, ciascuno con il proprio GIL. L'attivazione di questa funzionalità avviene esclusivamente tramite C API, consentendo:

- Avvio diretto di interpreti via libpython;
- Integrazione trasparente tra codice C++ e Python;
- Parallelismo reale senza overhead di processi multipli.

L'impossibilità di condividere oggetti tra interpreti, uno dei limiti di questo approccio, non costituisce un problema nel nostro caso poiché, grazie al meccanismo di smistamento dei pacchetti descritto nel prossimo paragrafo, ciò non sarà necessario.

L'eseguibile C++ una volta avviato, sarà in attesa del codice python da utilizzare come filtro: una volta arrivato il codice, questo viene compilato in bytecode

[51] `https://peps.python.org/pep-0684/`

tramite la funzione `compile` integrata in python e successivamente, al fine di rendere duplicabile il bytecode compilato nei vari interpreti inizializzati, ne eseguiamo il marshaling[52], ottenendone una rappresentazione serializzata. Come mostrato nel Codice 3.1, i thread designati all'analisi e al filtraggio delle connessioni, nella loro inizializzazione eseguiranno le procedure indicate nella documentazione delle C API di Python per creare un GIL indipendente da quello principale, e poi deserializzeranno il bytecode del filtro in python. Per nuovi aggiornamenti del filtro i thread si occuperanno deserializzare il nuovo bytecode una volta disponibile.

```
// Configurazione interprete con GIL indipendente
PyInterpreterConfig py_thread_config = {
    .use_main_obmalloc = 0,
    .allow_fork = 0,
    .allow_exec = 0,
    .allow_threads = 0,
    .allow_daemon_threads = 0,
    .check_multi_interp_extensions = 1,
    .gil = PyInterpreterConfig_OWN_GIL,
};

void before_loop() {
    PyStatus pystatus;
    tstate = PyThreadState_New(PyInterpreterState_Main());
    PyEval_AcquireThread(tstate); // Acquisizione GIL
    // Creazione del GIL indipendente + rilascio di quello principale
    pystatus = Py_NewInterpreterFromConfig(&tstate, &py_thread_config);
    handle_packet_code = unmarshal_code(...); // Deserializzazione bytecode
}
```

Codice 3.1: Inizializzazione di un processo consumatore con GIL indipendente in nfproxy.

Per ogni nuova connessione TCP inoltre, vengono eseguite le seguenti operazioni:

- Viene inizializzato un nuovo contesto globale Python dedicato a questa connessione;
- Viene eseguito il bytecode su questo contesto per inizializzare i filtri;
- Continuiamo a usare contesto inizializzato per eseguire le procedure di analisi e filtraggio del traffico (nel contesto sono inclusi i buffer necessari al *parsing*);
- Al termine della connesione, deallochiamo il contesto globale creato.

[52] https://docs.python.org/3/library/marshal.html

L'utilizzo delle funzionalità offerte dal PEP 684[51], soffre tuttavia di forti limitazioni sull'utilizzo di moduli senza multi-phase initialization[53] (includendo tutti quelli scritti in Cython[54]) e moduli con componenti non *thread-safe*. Nel corso dello sviluppo, queste limitazioni sono state oggetto di diverse problematiche, affrontate e approfondite in seguito.

Un'altra possibilità considerata è stata l'utilizzo dell'interprete in *free-threaded* mode di Python 3.13[55], attualmente sconsigliato per:

- Lo stato sperimentale della feature;
- Calo prestazionale dovuto alla disattivazione temporanea di alcuni sistemi di ottimizzazione del linguaggio;
- Incompatibilità con molti moduli CPython.

3.3.2 Bilanciamento del carico

Per il bilanciamento del carico tra i processi consumatori, finalizzato all'isolamento e alla distribuzione equa dei pacchetti, è stato implementato un meccanismo di hashing basato su indirizzi IP e porte di origine/destinazione mostrato nel Codice 3.2.

```
uint32_t hash_stream_id(const stream_id &sid) {
    uint32_t addr_hash = 0;
    addr_hash ^= min_addr[0] ^ min_addr[1] ^ min_addr[2] ^ min_addr[3];
    addr_hash ^= max_addr[0] ^ max_addr[1] ^ max_addr[2] ^ max_addr[3];
    uint32_t ports = (static_cast<uint32_t>(sid.min_address_port) << 16) |
    ↪  sid.max_address_port;
    uint32_t hash = addr_hash ^ ports;
    hash *= 0x9e3779b9; // Entropia nell'hashing
    return hash;
}
void balancer(PktRequest<Worker>* pkt) {
    const size_t idx = hash_stream_id(pkt->sid) % pkt->ctx->size();
    converted_pkt->ctx->queue.put(converted_pkt);
}
```

Codice 3.2: Funzione di hashing per stream_id (valida per IPv4/IPv6) in nfproxy.

Questo approccio garantisce una distribuzione uniforme dei pacchetti tra i consumatori, evitando sovraccarichi e assicurando un'elaborazione parallela efficiente,

[51] https://peps.python.org/pep-0684/

[53] https://peps.python.org/pep-0489/

[54] https://cython.org/

[55] https://docs.python.org/3/whatsnew/3.13.html#whatsnew313-free-threaded-cpython

oltre che a instradare i pacchetti di una stessa connessione sempre verso lo stesso thread. Il seguente meccanismo è stato re-integrato anche nel modulo `nfproxy` e anche progettato anche per essere facilmente utilizzato in futuri moduli di Firegex.

La tecnica riprende i meccanismi di bilanciamento usati kernel space da nfqueue, che tuttavia non considera le porte di origine/destinazione, limitandosi ad un hashing basato solo sugli indirizzi IP come mostrato nel Codice 3.3. Ciò è dovuto alla necessità di adattare l'algoritmo anche a protocolli diversi da TCP e UDP, come Internet Control Message Protocol (ICMP) che non prevede porte.

```
u32 nfqueue_hash(sk_buff *skb, u16 queue, u16 queues_total, u8 family,
    u32 initval) {
    switch (family) {
    case NFPROTO_IPV4:
        queue += reciprocal_scale(hash_v4(ip_hdr(skb), initval), queues_total);
        break;
    case NFPROTO_IPV6:
        queue += reciprocal_scale(hash_v6(ipv6_hdr(skb), initval),
        ↪  queues_total);
        break;
    }
    return queue;
}
```

Codice 3.3: funzione nfqueue_hash dal kernel Linux (semplificata) (versione 6.14-rc7).

L'esclusione delle porte nell'hashing kernel[56] nel sistema di *balancing*[57] integrato in nfqueue rappresenta un limite critico in scenari CTF con NAT, dove tutto il traffico anonimizzato verrebbe gestito da un singolo consumatore. Questo è il motivo che ha spinto alla scelta di un approccio più sofisticato, che garantisce un effettivo bilanciamento del carico sui vari thread.

Alcuni benchmark eseguiti sul sistema di bilanciamento sono disponibili nel Capitolo 4.3

3.4 Parsing dei pacchetti L3

All'arrivo dei pacchetti nella nfqueue, viene eseguita un'analisi immediata del protocollo di livello 3 mediante ispezione dei primi 4 bit del campo version, valida sia per IPv4 che IPv6. Identificata la versione IP, il *parsing* multilivello viene delegato a libtins[28], che ricostruisce ricorsivamente gli header di rete e trasporto.

[56] `nfqueue_hash`, hash function for `nf_queue` in the Linux kernel

[57] `nfqueue_tg_v3`, balancing function for `nf_queue` in the Linux kernel

[28] `https://libtins.github.io/`

Anche questa parte, come per il bilanciamento del carico, è stata progettata per essere facilmente utilizzata in futuri moduli di Firegex, e re-integrata anche in questo caso nel modulo `nfregex` (che supporta anche i pacchetti UDP, che pertanto vengono considerati in questo paragrafo).

La libreria genera automaticamente lo stream_id, struttura dati univoca per identificare connessioni TCP/UDP attraverso una coppia di tuple (IP:porta) sorgente/destinazione, che sono ordinate considerando la singola tupla come unica entità, e quindi non slegando mai IP e porta tra loro. Questo approccio permette di identificare univocamente una connessione, indipendentemente dalla direzione del traffico (ingresso/uscita). La fase di analisi del pacchetto tramite libtins[28] è eseguita dal costruttore della classe `PktRequest` mostrato nel Codice 3.4.

```
PktRequest(const char* payload, size_t plen, T* ctx, mnl_socket* nl,
           nfgenmsg *nfg, nfqnl_msg_packet_hdr *ph, bool is_input):
    ctx(ctx), nl(nl), res_id(nfg->res_id),
    packet_id(ph->packet_id), is_input(is_input),
    packet(string(payload, plen)),
    action(FilterAction::NOACTION),
    is_ipv6((payload[0] & 0xf0) == 0x60) // Verifica versione IP
{
    if (is_ipv6){
        ipv6 = new Tins::IPv6((uint8_t*)packet.c_str(), plen); // Parsing IPv6
        sid = stream_id::make_identifier(*ipv6); // Generazione stream_id
        _original_size = ipv6->size();
    }else{
        ipv4 = new Tins::IP((uint8_t*)packet.c_str(), plen); // Parsing IPv4
        sid = stream_id::make_identifier(*ipv4);
        _original_size = ipv4->size();
    }
    l4_proto = fill_l4_info(); // Estrazione info layer 4
}
```

Codice 3.4: Costruttore di PktRequest ed elaborazione del pacchetto di rete in firegex.

Inoltre, lo stream_id è indipendente dal protocollo internet utilizzato, identificando anche connessioni miste IPv4/IPv6 (usando indirizzi IPv6 *IPv4-Compatible*): questo permette di utilizzare la stessa queue per interfaccie di rete differenti, caratteristica potenzialmente utile per implementare nuove feature sulla selezione del traffico da inoltrare alla queue.

Ogni produttore utilizza questo dato come chiave per individuare i dati relativi ad una specifica connessione, salvandoli in una std::map[58] diversa per ogni singolo

[28] https://libtins.github.io/

[58] https://en.cppreference.com/w/cpp/container/map

produttore, così implementabile per merito del meccanismo di smistamento dei pacchetti descritto nel Capitolo 3.3.2. Il produttore, ricevuto il pacchetto, lo ingloba in un oggetto PktRequest, che contiene tutte le informazioni necessarie per l'elaborazione del pacchetto, e lo inserisce nella coda condivisa del produttore di riferimento. Sarà il produttore che una volta elaborato il pacchetto ed inviato il verdetto al kernel, deallocherà questa struttura passando all'elaborazione del prossimo pacchetto. Nessuna copia in memoria è pertanto necessaria in questo passaggio.

3.5 Gestione dei pacchetti TCP

Prima di inizializzare la queue, vengono (dal backend python) inserite tramite nftables le regole per intercettare il traffico in un certo punto della catena di netfilter, per essere mandato alla queue che determinerà un verdetto sul pacchetto.

Il traffico in particolare viene intercettato sia in `pre-routing`, che in `post-routing` ad un hook prima dell'azione del modulo `conntrack`, ma dopo dell'azione di defrag, rispettivamente a `prio` -310 in `pre-routing` e 110 in `post-routing` rientrando nei range corretti, come riportati sulla wiki ufficiale di nftables[59]. Questo permette di ottenere dei pacchetti che hanno già subito il processo di deframmentazione a livello 3, di cui pertanto non ci dovremo preoccupare, ma che non hanno subito il processo di routing (che quindi volendo potremmo manipolare) e che non sono stati ancora analizzati dal modulo `conntrack`. Tuttavia, qui i pacchetti non hanno subito il processo di ordinamento dei payload che avviene a livello 4, sul quale come è possibile osservare dalla Figura 2.2, non è possibile intercettare del traffico, poiché i pacchetti arriveranno direttamente alle fasi di gestioni dei dati per la loro consegna alla socket a livello applicatvo. Pertanto è dato a noi il compito di analizzare il traffico, ordinando i pacchetti TCP per un'utilizzo dei dati coerente a quelli che saranno letti dall'applicazione. Libtins semplifica significativamente questa gestione attraverso meccanismi automatizzati gestiti dal suo TCP follower[60] per:

- Chiusura dei flussi TCP basata su:
  - Analisi del traffico (flag FIN/RST);
  - Timeout configurabili (rilevamento di inattività);
  - Utilizzo eccessivo dei buffer di memoria.
- Ricostruzione dello stream da segmenti non ordinati (o duplicati);
- Gestione automatica della memoria utile alla ricostruzione.

[59] `https://wiki.nftables.org/wiki-nftables/index.php/Netfilter_hooks`

[60] Esempio TCP Follower: `https://libtins.github.io/examples/http-requests/`

Il follower inoltre permette facilmente l'integrazione di dati gestiti esternamente, tramite il suo sistema di callback, che ne chiamerà le funzionalità compatibilmente con quanto fatto con i buffer internamente. Le callback permettono di gestire le seguenti operazioni:

- Inizializzazione contesti;
- Pulizia risorse;
- Propagazione di eventi critici (per chiusure anomale dei flussi).

Ogni processo consumatore ha un suo follower dedicato che analizzerà i flussi relativi alla porzione traffico a lui dedicata.

3.6 Modifica dei pacchetti

Il modulo nfqueue consente effettivamente la modifica dei pacchetti, mediante l'invio al kernel Linux del pacchetto modificato, offrendo la possibilità di alterare header IP, TCP ed il relativo payload. Tuttavia la funzionalità di modifica in `nfproxy` è stata implementata solo parzialmente e non risulta pienamente affidabile.

Le limitazioni derivano da problematiche intrinseche legate all'interazione con i meccanismi di livello di trasporto. Ad esempio, modifiche al payload TCP durante connessioni attive potrebbero invalidare checksum o causare inconsistenze nei Sequence Number (SEQ). La complessità per garantire coerenza in tutti i possibili scenari (come in caso di ritrasmissioni) rende l'implementazione robusta di questa feature particolarmente onerosa. Inoltre, se gestita male, la modifica del payload potrebbe essere soggetta a vulnerabilità di sicurezza. Valutando il rapporto rischio-beneficio, si è optato per comunque rendere disponibile la funzionalità, seppur parzialmente implementata, ma fortemente disincentivata e volutamente resa meno facilmente accessibile.

L'effettiva necessità operativa risulta infatti marginale nella maggior parte degli scenari CTF, mentre i potenziali effetti collaterali verrebbero a compromettere uno degli obiettivi principali: la stabilità del servizio.

Di seguito si riporta una descrizione accurata delle problematiche riscontrate e delle soluzioni proposte, o applicate con relativi limiti e valutazioni rischio/beneficio. Le problematiche sul calcolo dei campi di integrità dei dati non sono trattate, ma considerate già risolte tramite l'utilizzo di libtins, che si occupa durante la serializzazione del pacchetto di ricalcolare questi campi nei vari layer interessati, pertanto non considerata una problematica rilevante.

L'unica operazione aggiuntiva affrontata su questo, è stata la necessità di reinserire il payload del pacchetto nella catena di Protocol Data Unit (PDU) di libtins, poiché (seppur non documentato), il follower del flusso TCP internamente separa

di fatto il payload dal pacchetto[61], rendendone necessario il reinserimento per la corretta serializzazione e calcolo di questi campi.

3.6.1 Cambio di dimensione del payload

Come emerge dalle specifiche stesse del protocollo di trasporto TCP[37], questo tiene traccia dello stato della corretta recezione dei pacchetti tramite un meccanismo di piggybacking che fa utilizzo dei campi numerici Acknowledgment Number (ACK) e SEQ. Nella trasmissione, il SEQ indica il byte da cui inizia il payload del pacchetto relativamente alla sua posizione nello stream, mentre il campo ACK della risposta di conferma proveniente dal ricevente, indica fino a che punto il flusso di dati è stato ricevuto correttamente.

Questo meccanismo è fondamentale per il corretto funzionamento del protocollo e per la corretta ricostruzione del flusso originale: di conseguenza la modifica alla dimensione del payload causa inevitabilmente l'incoerenza di questi valori che porteranno a comportamenti inaspettati, non conformi al normale funzionamento del protocollo.

Per risolvere la problematica descritta, è stato implementato un sistema di traduzione bidirezionale dei campi SEQ e ACK.

Questo meccanismo tenta di mantenere la coerenza logica della comunicazione riconvertendo i valori modificati in modo da preservare la corrispondenza tra la dimensione originale dei pacchetti e le aspettative del client (e viceversa del server). Il kernel e il TCP Follower di libtins ricevono così numerazioni coerenti con le alterazioni applicate, mantenendo la sincronizzazione degli stati di connessione. Il sistema opera in modo simmetrico per entrambe le direzioni del traffico: le modifiche applicate ai pacchetti in uscita vengono compensate da una riconversione inversa per quelli in entrata. L'attivazione avviene automaticamente alla prima modifica di un pacchetto nella sessione TCP e rimane attiva per tutta la durata della connessione, garantendo la persistenza del contesto di traduzione.

```
size_t payload_offset = data_size() - _data_original_size;
if (is_input){ // Inbound packets
    ack_seq_offset->in += payload_offset;
}else{ // Outbound packets
    ack_seq_offset->out += payload_offset;
}
```

Codice 3.5: Calcolo dell'offset cumulativo nello stream TCP da correggere in modifica.

[61] Libtins, istruzione per la separazione del payload nel follower (std::move del payload)

[37] RFC9293, Transmission Control Protocol (TCP)

Questa conversione avviene tramite il calcolo incrementale dell'offset che intercorre tra la dimensione del flusso originale e la dimensione del flusso modificato come mostrato nel Codice 3.5.

A seguito della modifica, nei prossimi pacchetti vengono compatibilmente ricalcolati gli ACK e SEQ, su tutto il traffico come mostrato nel Codice 3.6.

```
if (is_input){ // Inbound packets
    tcp->seq(tcp->seq() + ack_seq_offset->in);
    tcp->ack_seq(tcp->ack_seq() - ack_seq_offset->out);
}else{ // Outbound packets
    tcp->ack_seq(tcp->ack_seq() - ack_seq_offset->in);
    tcp->seq(tcp->seq() + ack_seq_offset->out);
}
```

Codice 3.6: Algoritmo di traduzione di ACK e SEQ numbers a seguito di mangle dei pacchetti.

Un'analisi preliminare e dei test preliminari, suggerirebbero l'adeguatezza di questa soluzione. Tuttavia, a seguito di analisi più accurate, si evince come il meccanismo presenti limitazioni intrinseche in determinate casistiche di traffico. L'approccio risulta efficace esclusivamente in condizioni ideali di trasmissione, dove non si verificano fenomeni di ritrasmissione di pacchetti.

La criticità principale risiede nell'algoritmo di conversione, che non considera adeguatamente le ritrasmissioni di segmenti antecedenti al pacchetto modificato. In tali scenari, il ricalcolo dell'ACK relativo richiederebbe l'utilizzo di un offset compatibile non all'ultimo offset calcolato, ma allo stato del flusso nella parte interessata dalla ritrasmissione, operazione che l'implementazione corrente non esegue. La problematica è diretta conseguenza della mancanza di un analisi del traffico contestuale, capace di identificare la porzione di flusso effettivamente coinvolta nella ritrasmissione, e di memorizzare le discrepanze cumulative associandole direttamente alle varie parti dello stream interessate.

L'analisi della Figura 3.3 evidenzia come le casistiche descritte possano compromettere integralmente il sistema di traduzione implementato. Tali scenari, particolarmente probabili in condizioni di congestione di rete, rappresentano potenziali vettori d'attacco mirati a destabilizzare il servizio. Alla luce di queste considerazioni, si conclude che la soluzione proposta non possiede la robustezza necessaria per un'implementazione definitiva. La funzionalità di modifica pacchetti viene pertanto mantenuta come opzione sperimentale, accompagnata da avvertenze esplicite sul suo comportamento instabile.

Una soluzione teoricamente più solida richiederebbe la memorizzazione di informazioni aggiuntive riguardo i vari punti del flusso dove il traffico è stato modificato, insieme agli offset applicati in quegli specifici intervalli, per tutta la finestra di trasmissione ancora non riscontrata dal ricevente. Inoltre, sarebbe necessaria una logica

di traduzione più complessa per gli ACK/SEQ, che vada ad esguire un calcolo coerente di questi valori, basato sul posizionamento che il segmento ha in trasmissione o riscontro nel flusso complessivo, sfruttando le nuove informazioni memorizzate.

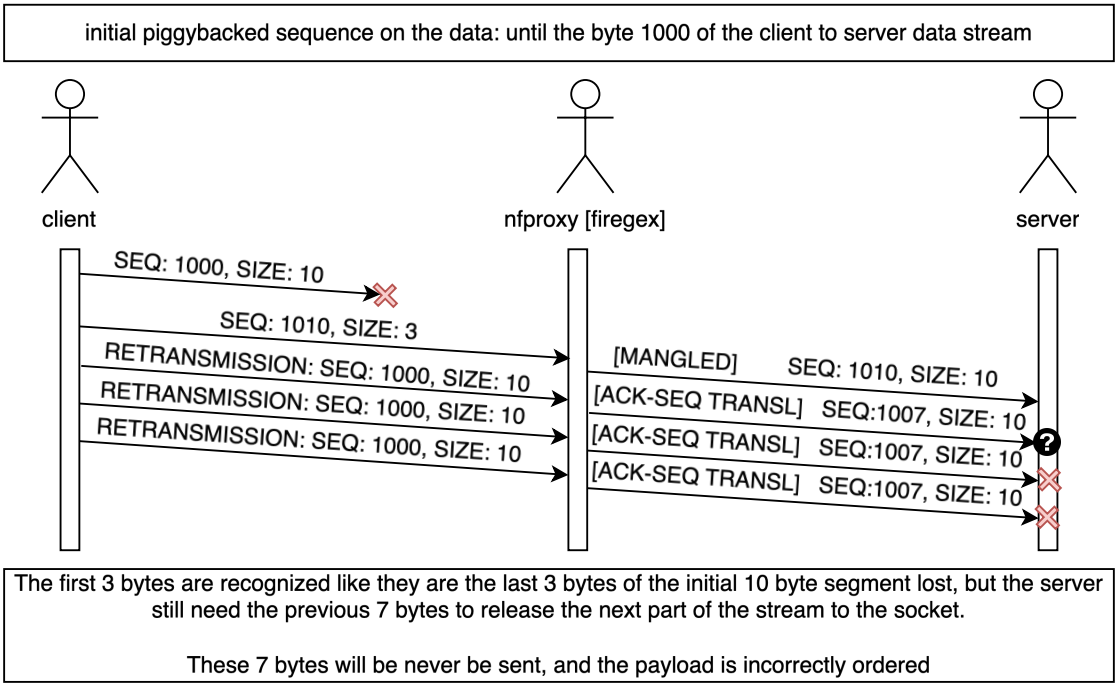

Figura 3.3: Esempio di una possibile problematica nella traduzione di ACK e SEQ implementata.

Considerando il rapporto costo-beneficio e il limitato valore operativo della funzionalità, si è optato per mantenere lo stato corrente dello sviluppo, consapevoli delle possibili criticità che può portare, non essendo una soluzione definitiva.

3.6.2 Modifica di segmenti non ordinati

Un'ulteriore criticità emerge nella gestione dei pacchetti con payload *out-of-order* da modificare. Nell'implementazione corrente, i pacchetti non ordinati vengono analizzati dal TCP Follower di libtins per la ricostruzione dello stream ed accettati immediatamente dopo questa elaborazione, privando tuttavia il sistema della possibilità di modificarli successivamente. Questo meccanismo, sebbene un payload malevolo venga accettato, riesce comunque a bloccarlo a livello applicativo, prima che il flusso possa essere completamente riordinato, intervenendo sul pacchetto con la prima parte del payload non ordinato, che normalmente andrebbe a causare il riempimento del buffer del socket a livello applicativo, ma che in questo caso causerà, grazie all'intervento di firegex, un blocco della connessione TCP.

Una soluzione alternativa prevedrebbe il buffering dei pacchetti *out-of-order* in attesa dei segmenti mancanti, applicando modifiche retroattive sui pacchetti trattenuti. Tuttavia, questo approccio introdurrebbe: una complessità significativa nel meccanismo di ricostruzione distribuita del flusso e il rischio di saturare la nfqueue kernel-side, con conseguente perdita di pacchetti e possibile esposizione ad attacchi DoS mirati a sfruttare le debolezze di questo sistema, potenzialmente portandolo ad un blocco del traffico non desiderato.

Sulla base di queste considerazioni, si conclude che il riscontro immediato dei pacchetti è un approccio che deve essere applicato al fine di scongiurare possibili criticità riguardanti il riempimento della coda kernel, soggetta a questa tipologia di attacchi. La scelta progettuale attuale, seppur limitante, garantisce un'operatività ed un throughput stabile e prevedibile, mantiene il sistema immune a possibili vulnerabilità ma non offre la possibilità di eseguire modifiche su pacchetti non ordinati.

Sarebbe possibile progettare un sistema che eviti i problemi di saturazione della coda kernel, implementando ad esempio un meccanismo basato su dei timeout che evitino la saturazione della coda eliminando i pacchetti potenzialmente fraudolenti. Tuttavia simili implementazioni richiederebbero un'attenta analisi ed uno studio molto più approfondito rispetto a quanto fatto finora, considerando anche un vasto spettro di casistiche insidiose che non si ritiene necessario affrontare in questa tesi e ai fini dello sviluppo di questo modulo.

3.7 Elaborazione dei pacchetti HTTP

Dall'analisi delle soluzioni esistenti per il *parsing* di pacchetti HTTP è emersa l'assenza di strumenti compatibili con i requisiti specifici del progetto, in particolare per quanto riguarda l'integrazione con le funzionalità introdotte dal PEP 684[51]. La necessità di supportare l'inizializzazione multi-fase (multi-phase initialization[53]) e garantire thread safety ha reso indispensabile un approccio personalizzato.

La soluzione adottata si basa su un fork dell'implementazione del wrapper python di Derrick Lyndon Pallas[62], del parser llhttp[63] sviluppato per Node.js. Questo fork, pubblicamente disponibile su GitHub[64] e distribuito tramite Python Package Index (PyPI), va a risolvere le limitazioni tecniche derivanti dalla sua versione originale e

[51] `https://peps.python.org/pep-0684/`

[53] `https://peps.python.org/pep-0489/`

[62] Python wrapper per llhttp: `https://github.com/pallas/pyllhttp`

[63] `https://github.com/nodejs/llhttp`

[64] `https://github.com/domysh/pyllhttp`

preserva le capacità core di llhttp, tra cui il *parsing* zero-copy dei dati, e la validazione rigorosa del protocollo HTTP/1.1[35].

3.7.1 Supporto agli algoritmi di compressione in HTTP

Nel traffico HTTP sono spesso in uso algoritmi di compressione per ridurre la dimensione del body. Dato il loro ampio utilizzo, al fine di semplificare l'implementazione di filtri che ne richiederebbero l'analisi del contenuto, si è deciso si supportare la decompressione automatica degli algoritmi supportati dal browser chrome:

- **zstd**[38]: Nativamente supportato in python;
- **brotli**[40]: supportato tramite un'implementazione eseguita da google[65], che tuttavia non supporta il PEP 684, per cui anche in questo caso è stato sviluppato un fork compatibile disponibile su github[66];
- **zstd**[41]: supportato tramite un wrapper python sviluppato da Sergey Dryabzhinsky[67], che tuttavia, essendo anch'esso non compatibile con il PEP 684, è stato personalizzato in un ulteriore fork[68];
- **deflate**[39]: Implementabile tramite il modulo zlib, nativamente supportato.

L'implementazione è stata scritta seguendo le specifiche definite nell'RFC 2616 il quale definisce la possibilità di compressioni multiple, correttamente supportate nell'implementazione finale.

3.7.2 Supporto alle websocket

Nell'implementazione per HTTP/1.1 è stato incluso anche il supporto alle websocket[36], tramite l'utilizzo della principale libreria python utilizzata per le decodifica dei Frame, ovvero websockets[69]. Da essa sono state prese le funzionalità di decodifica

[35] RFC2616, Hypertext Transfer Protocol – HTTP/1.1

[38] RFC1952, GZIP file format specification version 4.3

[40] RFC7932, Brotli Compressed Data Format

[65] `https://github.com/google/brotli`

[66] `https://github.com/domysh/brotli`

[41] RFC8478, Zstandard Compression and the application/zstd Media Type

[67] `https://github.com/sergey-dryabzhinsky/python-zstd`

[68] `https://github.com/domysh/python-zstd`

[39] RFC1951, DEFLATE Compressed Data Format Specification version 1.3

[36] RFC6455, The WebSocket Protocol

[69] `https://websockets.readthedocs.io/en/stable/`

dei frame, e le funzionalità per il supporto all'estensione permessage-deflate[42], per permetterne la decompressione, utilizzando internamente zlib. La complessità nell'implementazione delle websocket si è concentrata sulla necessità di assemblare il flusso di dati correttamente, dato il mancato supporto nativo al *parsing* in streaming, come invece supportato su llhttp. Nella fase di upgrading della connessione HTTP, vengono riconosciuti i plugin abilitati, e viene automaticamente introdotta la gestione dei websocket che si occupa di decodificare i prossimi dati sulla connessione TCP.

In caso di fallimento nel *parsing* dei frame, i dati vengono resi disponibili per il filtraggio, semplicemente come byte grezzi, stesso meccanismo usato in caso di *upgrade* ad HTTP/2 (non supportato attualmente).

3.8 Sintassi e gestione dei filtri python

I filtri Python per `nfproxy` vengono definiti in file singoli (uno per servizio) contenenti funzioni annotate con il decoratore `@pyfilter`. Queste funzioni, denominate collettivamente pyfilters, accettano parametri tipizzati che rappresentano i dati estratti dal traffico di rete e restituiscono istruzioni di azione (ACCEPT, REJECT, DROP, chiamati *statement*) determinando il comportamento finale sul pacchetto.

L'esempio nel Codice 3.7 illustra un filtro che blocca le richieste HTTP contenenti la stringa 'curl' nell'header `User-Agent` e impedisce gli *upgrade* a WebSocket:

```
from firegex.nfproxy.models import HttpRequest
from firegex.nfproxy import pyfilter, REJECT

@pyfilter
def curl_filter(r: HttpRequest):
    if r.upgrading_to_ws:
        return REJECT  # Blocca upgrade a WebSocket
    if "curl" in r.user_agent:
        print(f"Tentativo accesso con curl: {r.user_agent}")
        return REJECT  # Blocca client curl
```

Codice 3.7: Esempio di un filtro python per il modulo nfproxy.

Il sistema si appoggia sulla libreria firegex[70], che fornisce sia i decoratori per la definizione dei filtri, sia i modelli dati per l'annotazione dei parametri. Durante la

[42] RFC7692, Compression Extensions for WebSocket

[70] `https://pypi.org/project/firegex/`

fase di compilazione, `nfproxy` analizza le signature delle funzioni, sullo stesso principio di validazione tipica di pydantic[71], per determinare automaticamente quali layer di protocollo analizzare (e.g., HTTP, TCP), eventualmente quali campi estrarre dal payload, attivando internamente una serie di meccanismi per riuscire a soddisfare le richieste dei filtri.

La Tabella 3.1 riassume tutti i modelli di dati (`datahandlers`) disponibili per la definizione dei filtri, specificando per ciascuno il protocollo di riferimento, il momento di attivazione e le principali proprietà accessibili.

Model	**Proto**	**Trigger**	**Principali Proprietà**
`RawPacket`	TCP	Ogni pacchetto	`data`, `is_input`, `is_ipv6`, `is_tcp`, `l4_size`, `raw_packet`, `l4_data` (modificabili)
`TCPInputStream`	TCP	Solo pacchetti in ingresso	`data`, `total_stream_size`, `is_ipv6` (stream completo)
`TCPOutputStream`	TCP	Solo pacchetti in uscita	`data`, `total_stream_size`, `is_ipv6` (stream completo)
`HttpRequest`	HTTP	Header completi e/o body completo	`url`, `method`, `headers`, `user_agent`, `body`, `body_decoded`, `upgrading_to_ws`, `ws_stream`
`HttpRequestHeader`	HTTP	Solo header completi	Come `HttpRequest` ma `body` sempre `None`
`HttpFullRequest`	HTTP	Solo richieste completi	Come `HttpRequest`
`HttpResponse`	HTTP	Header completi e/o body completo	`url`, `status_code`, `headers`, `content_encoding`, `body`, `body_decoded`, `upgrading_to_ws`, `ws_stream`
`HttpResponseHeader`	HTTP	Solo header completi	Come `HttpResponse` ma `body` sempre `None`
`HttpFullResponse`	HTTP	Solo risposte completi	Come `HttpResponse`

Tabella 3.1: Modelli di dati disponibili in nfproxy per la definizione dei filtri.

Tutti i modelli supportano decompressione automatica (gzip, brotli, deflate, zstd) e gestione delle WebSocket con estensione permessage-deflate. I modelli `Http*` includono funzioni di utilità come `get_header()` per accesso case-insensitive agli header e proprietà booleane per identificare upgrade di protocollo (`upgrading_to_h2`, `upgrading_to_ws`).

[71] `https://docs.pydantic.dev/latest/`

3.8.1 Gestione dei buffer di memoria

La gestione di flussi di traffico con payload di grandi dimensioni richiede un'attenzione particolare al consumo di memoria, specialmente in contesti ad alto throughput. `nfproxy` implementa un sistema di buffering che consente una dimensione massima predefinita di 1MB per ciascun tipo di `datahandler`, tentando di mantenere il corretto funzionamento dei parser anche in condizioni di sovraccarico. Infatti quando la soglia viene superata, il buffer viene automaticamente svuotato sebbene ciò possa comportare perdita di informazioni utili.

Il comportamento predefinito è configurabile attraverso variabili globali che permettono di adattare il bilanciamento tra consumo di risorse e completezza analitica. È possibile ridurre la dimensione massima dei buffer per ambienti memory-constrained o aumentarla per scenari che richiedono la ricostruzione completa di flussi complessi. In aggiunta, l'azione da intraprendere al superamento della capacità del buffer può essere personalizzata tra quattro opzioni: svuotamento controllato (FLUSH), accettazione passiva del traffico successivo (ACCEPT), rifiuto selettivo della connessione (REJECT), o drop dei pacchetti successivi (DROP). Nel Codice 3.8 è disponibile un esempio di configurazione personalizzata del sistema di gestione dei buffer di memoria.

```
from firegex.nfproxy import FullStreamAction

FGEX_STREAM_MAX_SIZE = 4096  # Limite dimensionale per tipo di datahandler
FGEX_FULL_STREAM_ACTION = FullStreamAction.REJECT  # Politica di overflow

# Configurazione ottimizzata per flussi medio-piccoli
# Connessioni che superano 4KB vengono chiuse preventivamente
```

Codice 3.8: Esempio di configurazione di un filtro nfproxy per l'ottimizzazione del consumo di memoria.

3.9 Proxy di simulazione

Per validare il corretto funzionamento dei filtri prima del deployment sul firewall, firegex integra un ambiente di simulazione basato su proxy TCP. Questo strumento, accessibile tramite il comando fgex `nfproxy`, consente di testare le regole in condizioni controllate riproducendo fedelmente l'architettura operativa del modulo reale, basata su un proxy TCP.

```
fgex nfproxy test_http.py domy.sh 80 --proto http
```

Codice 3.9: Comando per avviare il simulazione di nfproxy integrato nella CLI di firegex.

Il comando mostrato nel Codice 3.9, avvia un proxy sull'indirizzo specificato (in questo caso domy.sh:80) applicando i filtri definiti in test_http.py. Per verificare gli effetti che i filtri avrebbero sul nostro server, possiamo configurare il simulatore per inoltrare il traffico al nostro server reale, interagendoci però tramite la connessione aperta dal simulatore stesso. In questo modo possiamo testare l'effettiva applicazione delle regole senza compromettere il funzionamento del server nella rete di gara. Inoltre il simulatore supporta l'hot-reload dei filtri, rilevando automaticamente le modifiche ai file Python senza richiedere riavvii. Questo sistema permette di testare rapidamente le modifiche ai filtri, riducendo il tempo di sviluppo e semplificando il processo di debugging.

Si specifica che il simulatore si occupa unicamente di replicare a grandi linee il comportamento della parte c++ del modulo, ma utilizza esattamente lo stesso codice per eseguire il *parsing* e per l'esecuzione dei filtri, permettendo nella maggior parte dei casi di avere un comportamento pressochè identico a quello che si avrebbe in produzione. La Figura 3.4 illustra l'output del simulatore durante l'esecuzione di un filtro HTTP, mostrando anche l'auto-reload eseguito in seguito all'esecuzione di modifiche.

```
~/Desktop » fgex nfproxy -h                                                                    domysh@domysh-mac

 Usage: fgex nfproxy [OPTIONS] FILTER_FILE ADDRESS PORT

 Run an nfproxy simulation

╭─ Arguments ──────────────────────────────────────────────────────────────────────────────────────────────────╮
│ *    filter_file      TEXT     The path to the filter file [default: None] [required]                       │
│ *    address          TEXT     The address of the target to proxy [default: None] [required]                │
│ *    port             INTEGER  The port of the target to proxy [default: None] [required]                   │
╰──────────────────────────────────────────────────────────────────────────────────────────────────────────────╯
╭─ Options ────────────────────────────────────────────────────────────────────────────────────────────────────╮
│ --proto                  [tcp|http]  The protocol to proxy [default: tcp]                                    │
│ --from-address           TEXT        The address of the local server [default: None]                         │
│ --from-port              INTEGER     The port of the local server [default: 7474]                            │
│                -6                    Use IPv6 for the connection                                             │
│ --help         -h                    Show this message and exit.                                             │
╰──────────────────────────────────────────────────────────────────────────────────────────────────────────────╯

~/Desktop » fgex nfproxy test_py domy.sh 80 --proto http                                       domysh@domysh-mac
[INFO][nfproxy][observer] Listening for changes on /Users/domysh/Desktop/test_py
[INFO][nfproxy][listener] TCP proxy listening on 127.0.0.1:7474 and forwarding to → domy.sh:80
[INFO][nfproxy][listener] Accepted connection from 127.0.0.1:61859
[INFO][nfproxy][listener] Accepted connection from 127.0.0.1:61860
[WARNING][nfproxy][RELOADING] Proxy reload triggered
[INFO][nfproxy][listener] TCP proxy listening on 127.0.0.1:7474 and forwarding to → domy.sh:80
[WARNING][nfproxy][RELOADING] Proxy reload triggered
[INFO][nfproxy][listener] TCP proxy listening on 127.0.0.1:7474 and forwarding to → domy.sh:80
[INFO][nfproxy][listener] Accepted connection from 127.0.0.1:61874
[INFO][nfproxy][listener] Accepted connection from 127.0.0.1:61875
```

Figura 3.4: Simulazione di nfproxy tramite il comando fgex.

Capitolo 4

Testing e Benchmarking

Ai fini di verificare il corretto funzionamento del sistema e valutarne le prestazioni, sono stati sviluppati una serie di test e benchmark. Firegex già presentava un set di test, che con l'integrazione di `nfproxy` sono stati ampliati e migliorati, al fine di coprire anche le nuove feature introdotte.

I test sono eseguiti prima di ogni rilascio in automatico, al fine di garantire che le modifiche apportate non abbiano introdotto regressioni. I rilasci di firegex avvengono tramite GitHub Actions, che ad ogni rilascio su GitHub esegue i test e, in caso di successo, pubblica l'immagine del container su GitHub Container Registry (GHCR) (ghcr.io/pwnzer0tt1/firegex) e rilascia una nuova versione della libreria, inclusa nello stesso repository, su PyPI.

I rilasci avvengono sia per architetture x86_64 che ARM64, al fine di supportare il maggior numero di macchine possibile.

4.1 Integrazione dei test

Tutti i test di firegex sono disponibili nel suo repository, all'interno della cartella tests. In particolare, quelli riguardanti `nfproxy` sono contenuti nel file tests/nfproxy_test.py, ed esegue la seguente serie di operazioni:

- Avvia un nuovo servizio sull'interfaccia di loopback, e verifica che il traffico non sia stato compromesso dal semplice avvio del modulo;
- Verifica che ogni tipo di *statement* sia correttamente funzionante, inserendo un filtro che individua se sul segmento TCP sia presente un payload specifico, che se inviato, causi il comportamento desiderato, imposto dallo *statement* inserito;
- Verifica che la funzionalità di modifica funzioni correttamente (nelle casistiche in cui dovrebbe), eseguendo prove sia su mangle a pacchetti più piccoli, che a pacchetti più grandi, comprovando pertanto che la traduzione degli

ACK e SEQ avvenga correttamente. Inoltre viene verificato se a seguito della modifica, il traffico di rete continui a transitare normalmente;
- Si assicura che le funzionalità di interruzione del servizio e del singolo pyfilter siano correttamente funzionanti;
- Per ogni tipo di `datahandler`, si scambiano dei payload di prova al fine di verificare se il *parsing* dei vari dati avvenga correttamente per i model: TCPInputStream, TCPOutputStream, HttpRequest, HttpResponse, HttpRequestHeader, HttpResponseHeader includendo anche il *parsing* di Frame websocket;
- Vengono testate tutte le API backend di `nfproxy`, verificando che le chiamate restituiscano i valori corretti, e si comportino in modo coerente alla loro funzione;
- Si verifica inoltre che vengano conteggiate correttamente le connessioni bloccate, e i pacchetti modificati nelle statistiche.

4.2 Benchmark nfproxy

I benchmark sono stati realizzati tramite uno script python con l'ausilio di iperf3[72].

Le casistiche utilizzate sono di semplice natura: verificano le prestazioni in termini di throughput, e lo fanno sia a servizio attivo senza filtri, che con un filtro che applica la seguente regex al payload TCP (regex per il matching di email):

```
(?:[a-z0-9!#$%&'*+/=?^_`{|}~-]+
    (?:\.[a-z0-9!#$%&'*+/=?^_`{|}~-]+)* |
    "(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f] |
        \\[\x01-\x09\x0b\x0c\x0e-\x7f])*")
@
(?:(?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+
    [a-z0-9](?:[a-z0-9-]*[a-z0-9])? |
    \[(?:(?:25[0-5] | 2[0-4][0-9] | [01]?[0-9][0-9]?)\.){3}
        (?:25[0-5] | 2[0-4][0-9] | [01]?[0-9][0-9]? |
            [a-z0-9-]*[a-z0-9]:
            (?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f] |
                \\[\x01-\x09\x0b\x0c\x0e-\x7f])+)\])
```

A differenza dei test, i benchmark non hanno lo scopo di applicare i filtri su un traffico da bloccare, ma si inserisce la seguente regola all'unico scopo di misurare l'overhead causato dalla presenza del filtro stesso.

[72] `https://iperf.fr/`

I benchmark sono stati eseguiti su un Macbook Air M2 16GB RAM tramite una VM avviata da OrbStack con Fedora Linux 41 (Container Image) con kernel aarch64 Linux 6.12.13-orbstack-00304-gede1cf3337c4.

Lo script python che esegue i benchmark ripete la stessa misura un numero configurabile di volte, sia per il modulo `nfproxy`, sia per il modulo `nfregex`. I risultati sono stati raccolti e analizzati, e messi a disposizioni nei grafici in seguito, eseguiti con i comandi mostrati nel Codice 4.1.

```
python3 comparemark.py nfproxy -p testpassword -d 1 -s 50 -V 100
python3 comparemark.py nfregex -p testpassword -d 1 -s 50 -V 100
```

Codice 4.1: Comandi per l'esecuzione dei benchmark su nfproxy e nfregex

I benchmark pertanto sono stati eseguiti con 1 secondo di durata della singola misura, 50 connessioni simultanee, e 100 misure eseguite. Il numero indicato dopo la T nei grafici indica il numero di thread con cui firegex è stato configurato: quindi eseguiti sia in single-threading che in multi-threading (con 8 thread).

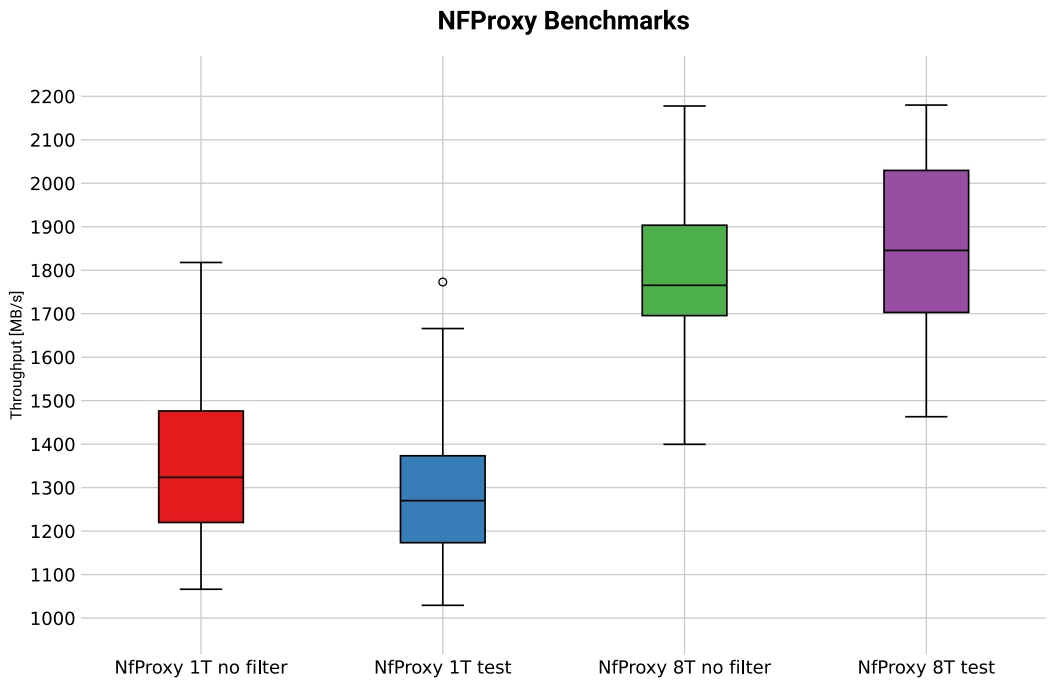

Figura 4.1: Grafico Whisker sulle misure di throughput di nfproxy.

Dalla Figura 4.1 si può notare come le performance in *multi-threading* di `nfproxy` siano migliori rispetto all'utilizzo del singolo thread. È importante sottolineare come

nei benchmark non viene eseguito alcun tipo di elaborazione riguardo il *parsing* dei dati, pertanto il carico di elaborazione è comunque limitato.

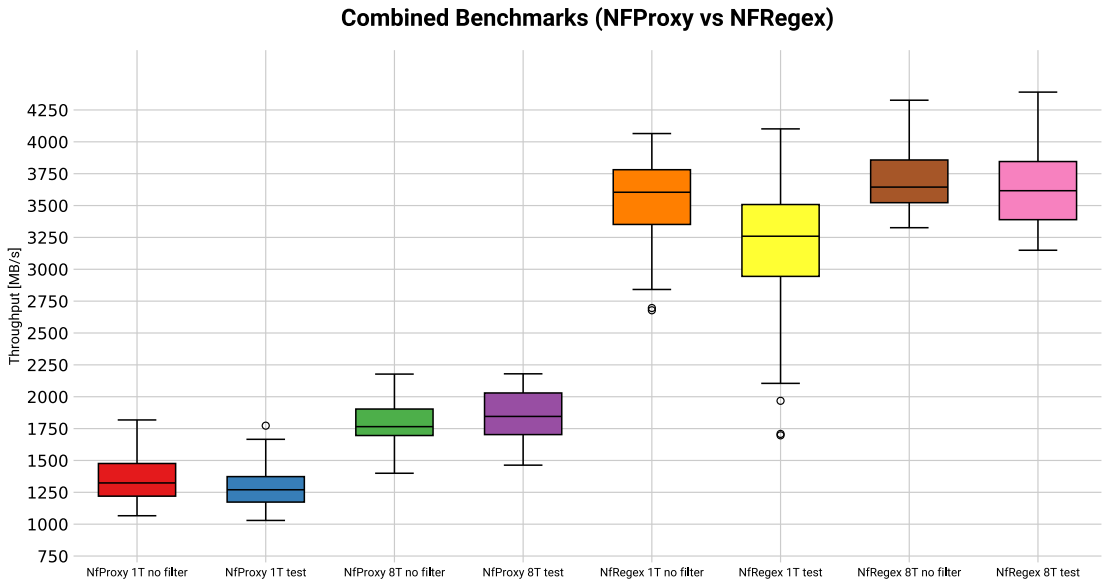

Figura 4.2: Grafico Whisker sulle misure di throughput di nfproxy e nfregex.

Dal confronto di nfproxy e nfregex in Figura 4.2 emerge chiaramente l'impatto prestazionale dell'introduzione di Python nel processo di elaborazione. Nella configurazione single-thread (senza carico), nfregex raggiunge un throughput medio di 3548 MB/s con una deviazione standard contenuta (ampiezza whisker di circa 375 MB/s), mentre nfproxy si attesta a 1357.16 MB/s con però una minore variabilità (ampiezza whisker di circa 250 MB/s).

Il passaggio alla configurazione multi-thread (8 thread) mostra miglioramenti significativi per entrambi i moduli, ma soprattutto per nfproxy avendo più carico di base da sostenere: nfregex sale a 3716.94 (+4.55% rispetto al single-thread) mantenendo una variabilità simile (300 MB/s circa), mentre nfproxy raggiunge 1816.05 MB/s (+25.27% rispetto al single-thread), dimostrando una maggiore capacità di beneficiare della parallelizzazione grazie al superamento delle limitazioni del GIL tramite il PEP 684.

Nonostante il calo prestazionale del 51.14% rispetto a nfregex nella configurazione multi-thread, nfproxy mantiene un throughput abbondantemente sufficiente per scenari CTF A/D.

4.3 Benchmark sul sistema di balancing

I benchmark sul sistema di balancing descritto nel Capitolo 3.3.2 sono stati eseguiti sul modulo `nfregex` con carico simulato (mediante il Codice 4.2), mostrano il

beneficio che porta questo sistema come visibile dai risultati in Figura 4.3.

```
volatile int x = 0;
for (int i=0; i<50000; i++){
    x+=1;
}
```

Codice 4.2: Algoritmo di simulazione del carico usato nei benchmark multi-thread di nfregex.

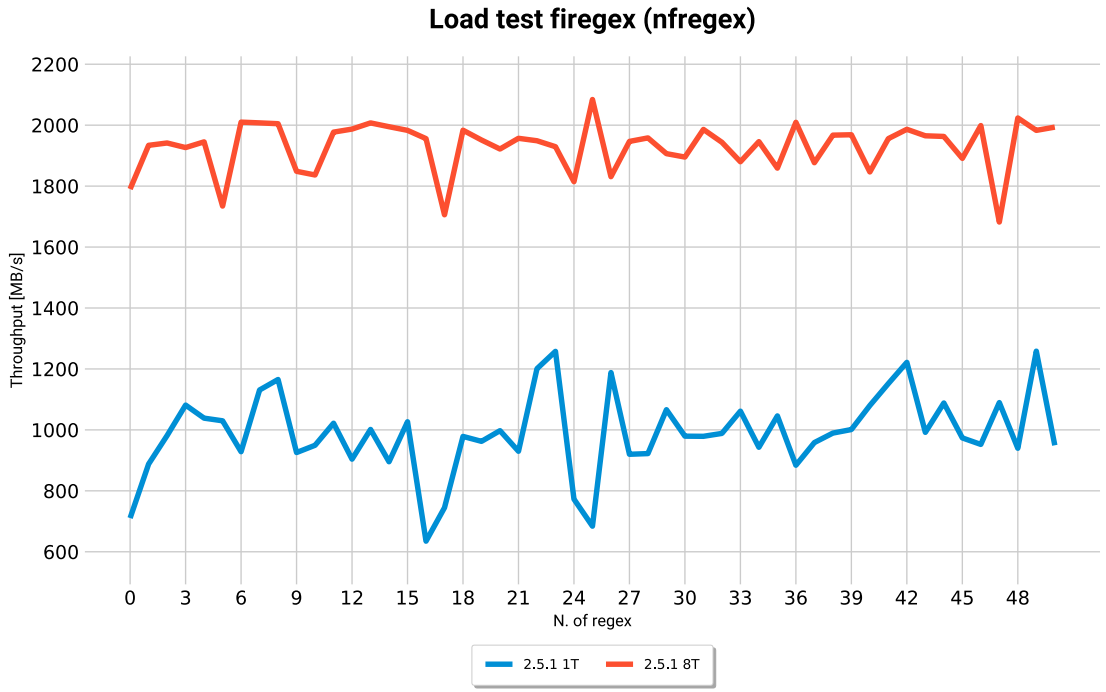

Figura 4.3: Benchmark di nfproxy con load balancing personalizzato (1 vs 8 thread).

I medesimi vantaggi prestazionali si possono riscontrare in `nfproxy`, che condivide la stessa base di codice per l'implementazione del bilanciamento del carico (come già mensionato nel Capitolo 3.3.2). Si noti come con questo carico simulato registriamo un throughput migliorato circa del 44% grazie al multithreading e al bilanciamento del carico.

Conclusioni

In questo lavoro di tesi è stata presentata la progettazione e lo sviluppo dell'SDFW Firegex, e nello specifico il modulo nfproxy. Quest'ultimo, permette rispetto ai moduli già esistenti di filtrare con un'elevata flessibilità e accuratezza il traffico, tramite l'implementazione di filtri in linguaggio Python.

Nel corso del lavoro sono stati descritti i dettagli architetturali del modulo, approfondite le scelte implementative delle funzionalità principali e illustrate le modalità di utilizzo pratico. Sono stati inoltre condotti specifici benchmark per valutare le prestazioni del sistema in diversi scenari operativi.

Nonostante il calo di performance, notoriamente previsto rispetto al modulo nfregex, nfproxy ha dimostrato avere prestazioni sufficienti per poter essere utilizzato in una competizione Attack-Defense. Questo è stato possibile grazie agli sforzi eseguiti nella progettazione e nell'implementazione dell'architettura, che ha permesso di ottenere ottimi risultati, portandolo a diventare uno strumento efficace per la difesa dei servizi.

Il lavoro fatto per questa tesi può essere inoltre un ottimo punto di partenza per eseguire ulteriori potenziamenti al modulo stesso, ma anche per l'implementazione di alternative che fanno uso dello stesso modello architetturale qui presentato.

Infatti, l'integrazione di nuove funzionalità renderebbe nfproxy un modulo ancora più versatile e flessibile di quanto non lo sia già, permettendone l'utilizzo in contesti ancora più ampi e diversificati. Tra le possibili evoluzioni future si potrebbero considerare l'estensione del supporto a protocolli aggiuntivi come HTTP/2 e UDP, l'implementazione di meccanismi avanzati per la condivisione di dati tra thread mediante strutture C thread-safe, e lo sviluppo di funzionalità di monitoraggio in tempo reale delle code di Netfilter attraverso le API kernel dedicate. Un'altra direzione promettente potrebbe essere la realizzazione di un'architettura alternativa basata su proxy Man In The Middle (MITM) tradizionali con translation trasparente di indirizzi e porte, che manterrebbe la stessa flessibilità di filtraggio ma con un approccio architetturale differente.

Concludendo, il lavoro svolto ha dimostrato come sia possibile coniugare flessibilità e prestazioni in un contesto altamente competitivo come quello delle competizioni CTF A/D, fornendo uno strumento che non solo risponde alle esigenze immediate, ma che può evolversi per affrontare sfide future.

Note: CVE-2022-36946

Durante lo sviluppo di `nfregex`, lavorando con un altro membro del team Pwnzer0tt1[7], Nicola Guerrera, è stata scoperta una vulnerabilità di sicurezza che permetteva a un utente malintenzionato di causare un crash del kernel.

La vulnerabilità è stata inizialmente segnalata al team di sicurezza del kernel, che ha rapidamente rilasciato una *patch*[73]. Successivamente è stata segnalata al MITRE, e quindi identificata come **CVE-2022-36946**[74].

In un primo momento la problematica ci è sembrata difficilmente sfruttabile, in quanto pensavamo richiedesse necessariamente i privilegi di amministratore per essere correttamente sfruttata. Tuttavia, la vulnerabilità è stata classificata come critica (7.5 HIGH), in quanto con l'utilizzo dei Linux namespace permette il crash del kernel anche tramite utenti non privilegiati. È possibile verificare e sfruttare la vulnerabilità, senza privilegi, tramite il *One-Command* mostrato nel Codice 4.3.

```
curl -sLf https://pwnzer0tt1.it/cve-2022-36946.sh | bash
```

Codice 4.3: One-Command tramite cui eseguire la POC della CVE-2022-36946.

Informazioni aggiuntive e il sorgente completo della Proof Of Concept (POC) sono disponibili sulla pagina `https://github.com/Pwnzer0tt1/CVE-2022-36946`.

[73] netfilter: nf_queue: do not allow packet truncation below transport header offset

[74] `https://nvd.nist.gov/vuln/detail/cve-2022-36946`

Ringraziamenti

Ringrazio innanzitutto il mio relatore, il Prof. Gennaro Boggia, per avermi supportato nell'analisi delle diverse problematiche emerse durante la progettazione, e per avermi concesso la possibilità di lavorare alla tesi da me proposta.
Ringrazio inoltre il mio correlatore, il Dott. Ing. Daniele Pugliese, per il prezioso supporto durante la stesura del testo e per i consigli e suggerimenti che si sono rivelati fondamentali per il completamento di questo lavoro.

Un ringraziamento speciale va alla mia famiglia: a mia madre, a mio padre e a mio fratello, per avermi sostenuto in questi tre anni, permettendomi di concentrarmi sul mio percorso di studi e supportandomi in ogni momento.

Un grande grazie va anche a tutti i miei amici:

- **Vanny**, per essermi stato vicino non solo durante il percorso universitario, ma anche lungo tutto il mio cammino di vita fino ad oggi. Grazie per l'entusiasmo, i viaggi, le risate, ma soprattutto per i consigli, i momenti di riflessione e di sfogo in cui ci siamo supportati e rafforzati a vicenda;

- **Federica**, per avermi rialzato nei momenti più bui, sopportandomi e supportandomi con un affetto e una vicinanza impareggiabili. Grazie per ogni momento passato assieme, per averci regalato attimi di serenità e leggerezza, dove ogni pensiero era benvenuto e mai giudicato (o quasi);

- Ai miei colleghi di studio, **Vito** e **Daniele**, per il supporto che è andato ben oltre la semplice collaborazione accademica, trasformandosi in un legame autentico che ha reso più leggere anche le lezioni e le giornate più impegnative;

- **Nicola (Guerrera)**, per il sostegno affettivo e per la collaborazione nei progetti, tra cui quello presentato in questa stessa tesi;

- **Gilberto**, che, seppur conosciuto da relativamente poco, mi ha dimostrato una vicinanza costante e un affetto genuino, a 2π radianti di calore umano;

- **Gianmarco**, per avermi sopportato ogni mattina durante il tragitto verso le lezioni, e per tutte le sessioni di studio condivise nella preparazione degli esami.

Un grazie va anche a **Brian, Claudio, Domenico, Fabio, Francesco, Gerardo, Gianfranco, Lara, Oronzo** e **Vito**: ognuno di voi ha contribuito a rendere questo percorso più leggero e piacevole, tra risate, momenti di pausa e persino quelle volte in cui mi avete zittito per fermare il mio irrefrenabile bisogno di parlare di informatica.

Infine, un sentito ringraziamento ai miei professori delle scuole superiori, che mi hanno fornito una solida preparazione, sia tecnica che personale, rivelatasi fondamentale per affrontare questo percorso e la vita stessa.

Non scriverò altre parole, ma spero che quelle già scritte siano sufficientemente profonde per esprimere la sincera gratitudine che provo nei vostri confronti.

Grazie!

Domingo Dirutigliano

Bibliografia

[1] Mark Elsner, Grace Atkinson, and Saadia Zahidi. Global risks report 2025, January 2025. 20th edition.

[2] Google. Google ctf. https://capturetheflag.withgoogle.com/. Visitato in data: 30/04/2025.

[3] Kim Crawley. What is ctf in hacking? tips & ctfs for beginners by htb. https://www.hackthebox.com/blog/what-is-ctf/. Visitato in data: 18/03/2025.

[4] Pwnzer0tt1 Domingo Dirutigliano. Firegex, an application firewall for attack-defense ctf. https://github.com/Pwnzer0tt1/firegex. Visitato in data: 18/03/2025.

[5] Politecnico di Bari. Politecnico di bari. https://www.poliba.it/. Visitato in data: 25/03/2025.

[6] Cybersecurity National Lab CINI. Cyberchallenge.it. https://cyberchallenge.it/. Visitato in data: 18/03/2025.

[7] Pwnzer0tt1 Domingo Dirutigliano. Pwnzer0tt1. https://pwnzer0tt1.it/. Visitato in data: 18/03/2025.

[8] Faust Team. Attack-defense for beginners. https://faustctf.net/information/attackdefense-for-beginners/. Visitato in data: 18/03/2025.

[9] Cybersecurity National Lab CINI. A/d ctf - cybersecurity national lab. https://ad.cyberchallenge.it/rules/. Visitato in data: 16/04/2025.

[10] Domingo Dirutigliano. Ctfbox, an attack-defense ctf infrastructure. https://github.com/domysh/ctfbox. Visitato in data: 18/03/2025.

[11] TheRomanXpl0it. Theromanxpl0it. https://theromanxpl0.it/about.html. Visitato in data: 18/03/2025.

[12] Wireshark. Wireshark. https://www.wireshark.org/. Visitato in data: 18/03/2025.

[13] Suricata. Suricata. https://suricata.io/. Visitato in data: 18/03/2025.

[14] Emiliano Ciavatta. Analisi e ricostruzione di flussi tcp con il software caronte. https://amslaurea.unibo.it/id/eprint/23321/1/tesi_caronte_eciavatta.pdf. Visitato in data: 16/06/2025.

[15] OpenAttackDefenseTools Team Europe. Tulip. https://github.com/OpenAttackDefenseTools/tulip. Visitato in data: 18/03/2025.

[16] Destructive Voice. Destructivefarm. https://github.com/DestructiveVoice/DestructiveFarm. Visitato in data: 18/03/2025.

[17] Pwnzer0tt1 Domingo Dirutigliano. Exploitfarm. https://github.com/Pwnzer0tt1/exploitfarm. Visitato in data: 18/03/2025.

[18] ByteLeMani. Ctf proxy. https://github.com/ByteLeMani/ctf_proxy. Visitato in data: 18/03/2025.

[19] Inc. GitHub. Cve-2022-23540. https://nvd.nist.gov/vuln/detail/cve-2022-23540. Visitato in data: 01/05/2025.

[20] Nginx. Nginx. https://nginx.org/. Visitato in data: 19/03/2025.

[21] Meta. React. https://reactjs.org/. Visitato in data: 19/03/2025.

[22] Vite Contributors VoidZero Inc. Vite. https://vite.dev/. Visitato in data: 16/06/2025.

[23] Sebastián Ramírez. Fastapi. https://fastapi.tiangolo.com/. Visitato in data: 19/03/2025.

[24] The Netfilter Team. nftables. https://netfilter.org/projects/nftables/. Visitato in data: 19/03/2025.

[25] The Netfilter Team. Netfilter queue. https://netfilter.org/projects/libnetfilter_queue/. Visitato in data: 19/03/2025.

[26] Docker. Docker. https://www.docker.com/. Visitato in data: 19/03/2025.

[27] The Netfilter Team. Nftables python library. https://git.netfilter.org/nftables/tree/py. Visitato in data: 19/03/2025.

[28] libtins. libtins. https://libtins.github.io/. Visitato in data: 19/03/2025.

[29] The Netfilter Team. libmnl. `https://netfilter.org/projects/libmnl/`. Visitato in data: 19/03/2025.

[30] VectorCamp. Vectorscan. `https://vectorcamp.gr/project/vectorscan/`. Visitato in data: 19/03/2025.

[31] Python Software Foundation. Python/c api reference manual. `https://docs.python.org/3/c-api/`. Visitato in data: 19/03/2025.

[32] Philip Hazel. Pcre. `https://www.pcre.org/`. Visitato in data: 19/03/2025.

[33] Xiang Wang, Yang Hong, Harry Chang, KyoungSoo Park, Geoff Langdale, Jiayu Hu, and Heqing Zhu. Hyperscan: A fast multi-pattern regex matcher for modern {CPUs}. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019.

[34] Python Software Foundation. Python. `https://python.org/`. Visitato in data: 19/03/2025.

[35] Henrik Nielsen, Jeffrey Mogul, Larry M Masinter, Roy T. Fielding, Jim Gettys, Paul J. Leach, and Tim Berners-Lee. Hypertext Transfer Protocol – HTTP/1.1. RFC 2616, June 1999.

[36] Alexey Melnikov and Ian Fette. The WebSocket Protocol. RFC 6455, December 2011.

[37] Wesley Eddy. Transmission Control Protocol (TCP). RFC 9293, August 2022.

[38] L. Peter Deutsch. GZIP file format specification version 4.3. RFC 1952, May 1996.

[39] L. Peter Deutsch. DEFLATE Compressed Data Format Specification version 1.3. RFC 1951, May 1996.

[40] Jyrki Alakuijala and Zoltan Szabadka. Brotli Compressed Data Format. RFC 7932, July 2016.

[41] Yann Collet and Murray Kucherawy. Zstandard Compression and the application/zstd Media Type. RFC 8478, October 2018.

[42] Takeshi Yoshino. Compression Extensions for WebSocket. RFC 7692, December 2015.

[43] University of Southern California Information Sciences Institute. Internet Protocol. RFC 791, September 1981.

[44] Bob Hinden and Dr. Steve E. Deering. Internet Protocol, Version 6 (IPv6) Specification. RFC 2460, December 1998.

[45] The Netfilter Team. Function documentation: nfq_nlmsg_verdict_put. https://netfilter.org/projects/libnetfilter_queue/doxygen/html/group__nfq__verd.html. Visitato in data: 21/03/2025.

[46] cppreference.com. std::mutex. https://en.cppreference.com/w/cpp/thread/mutex. Visitato in data: 21/03/2025.

[47] cppreference.com. std::condition_variable. https://en.cppreference.com/w/cpp/thread/condition_variable. Visitato in data: 21/03/2025.

[48] Michael Kerrisk. pipe(2) — linux manual. https://man7.org/linux/man-pages/man2/pipe.2.html. Visitato in data: 21/03/2025.

[49] Arif Jaffer. Blocking queue in c++ using a condition variable. https://www.bit-byter.com/blog/files/blocking-q-cpp.html. Visitato in data: 21/03/2025.

[50] Beazley David. Understanding the python gil. In *PyCON Python Conference. Atlanta, Georgia*, pages 1–62, 2010.

[51] Eric Snow. A per-interpreter gil. PEP 684, 2022.

[52] Python Software Foundation. marshal — internal python object serialization. https://docs.python.org/3/library/marshal.html. Visitato in data: 21/03/2025.

[53] Petr Viktorin, Stefan Behnel, and Alyssa Coghlan. Multi-phase extension module initialization. PEP 489, 2013.

[54] Stefan Behnel, Robert Bradshaw, David Woods, Matúš Valo, and Lisandro Dalcín. Cython. https://cython.org/. Visitato in data: 21/03/2025.

[55] Python Software Foundation. What's new in python 3.13 - free-threaded cpython. https://docs.python.org/3/whatsnew/3.13.html#whatsnew313-free-threaded-cpython. Visitato in data: 21/03/2025.

[56] Linus Torvalds. nfqueue_hash, hash function for nf_queue in the linux kernel. https://web.git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/net/netfilter/nf_queue.h?h=v6.14-rc7#n105. Visitato in data: 30/04/2025.

[57] Linus Torvalds. nfqueue_tg_v3, balancing function for nf_queue in the linux kernel. https://web.git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/net/netfilter/xt_NFQUEUE.c?h=v6.14-rc7#n86. Visitato in data: 30/04/2025.

[58] cppreference.com. std::map. https://en.cppreference.com/w/cpp/container/map. Visitato in data: 22/03/2025.

[59] The Netfilter Team. Netfilter hooks. https://wiki.nftables.org/wiki-nftables/index.php/Netfilter_hooks. Visitato in data: 22/03/2025.

[60] libtins. Libtins, esempio tcp follower: 'http requests'. https://libtins.github.io/examples/http-requests/. Visitato in data: 30/04/2025.

[61] libtins. Libtins, istruzione per la separazione del payload nel follower (std::move del payload). https://github.com/mfontanini/libtins/blob/b7e61f4c76ac64053c9c4c9f8eadaabbe3a9381a/src/tcp_ip/data_tracker.cpp#L68. Visitato in data: 30/04/2025.

[62] Derrick Lyndon Pallas. Python wrapper for llhttp. https://github.com/pallas/pyllhttp. Visitato in data: 22/03/2025.

[63] Node.js. llhttp: a port of http_parser to llparse. https://github.com/nodejs/llhttp. Visitato in data: 22/03/2025.

[64] Domingo Dirutigliano and Derrick Lyndon Pallas. Python wrapper for llhttp pep 489 compliant. https://github.com/domysh/pyllhttp. Visitato in data: 22/03/2025.

[65] Google. Brotli. https://github.com/google/brotli. Visitato in data: 26/03/2025.

[66] Domingo Dirutigliano and Google. Brotli fork with pep 489 compliant multi-phase initialization. https://github.com/domysh/brotli. Visitato in data: 22/03/2025.

[67] Sergey Dryabzhinsky. Zstandard python wrapper. https://github.com/sergey-dryabzhinsky/python-zstd. Visitato in data: 22/03/2025.

[68] Domingo Dirutigliano and Sergey Dryabzhinsky. Zstandard python wrapper with pep 489 compliant multi-phase initialization. https://github.com/domysh/python-zstd. Visitato in data: 22/03/2025.

[69] Aymeric Augustin. Websockets. https://websockets.readthedocs.io/en/stable/. Visitato in data: 22/03/2025.

[70] Pwnzer0tt1 Domingo Dirutigliano. Firegex on pypi. https://pypi.org/project/firegex/. Visitato in data: 22/03/2025.

[71] The Pydantic Community. Pydantic. https://docs.pydantic.dev/latest/. Visitato in data: 22/03/2025.

[72] Jon Dugan, Seth Elliott, Bruce A. Mah, Jeff Poskanzer, and Kaustubh Prabhu. iperf - the ultimate speed test tool for tcp, udp and sctp. https://iperf.fr/. Visitato in data: 23/03/2025.

[73] Florian Westphal. netfilter: nf_queue: do not allow packet truncation below transport header offset. https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=99a63d36cb3ed5ca3aa6fcb64cffbeaf3b0fb164. Visitato in data: 30/04/2025.

[74] Domingo Dirutigliano and Nicola Guerrera. Cve-2022-36946. https://github.com/Pwnzer0tt1/CVE-2022-36946, https://nvd.nist.gov/vuln/detail/cve-2022-36946. Visitato in data: 23/03/2025.